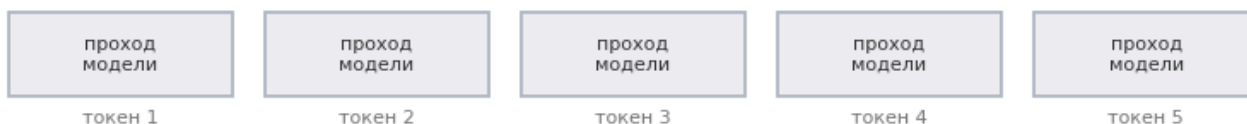


EAGLE-3 на Qwen3-1.7B: где спекулятивное декодирование ускоряет вдвое, а где замедляет

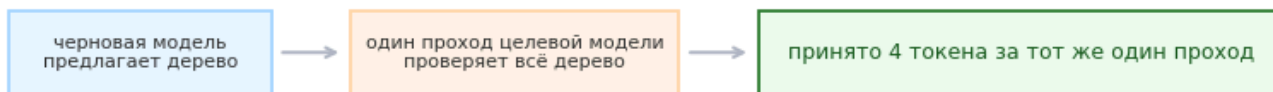
Автор: [Georgy Mamarin](#) · разбор статьи [EAGLE-3: Yuhui Li, Fangyun Wei, Chao Zhang, Hongyang Zhang, NeurIPS 2025](#)

Языковая модель пишет по одному токenu за раз, и каждый токен стоит отдельного прохода через все её веса. Спекулятивное декодирование обходит это так: маленькая черновая модель заранее предлагает несколько следующих токенов, большая целевая проверяет их разом за один проход и оставляет те, что совпали с её собственным выбором. Текст на выходе тот же — быстрее становится только его получение.

обычная генерация



спекулятивное декодирование



выдача при этом не меняется: проверку проходит только то, что целевая модель выбрала бы сама

Сверху — обычная генерация: пять проходов ради пяти токенов. Снизу — четыре из тех же пяти токенов за один проход целевой модели; пятый достанется следующему циклу. Сколько именно выйдет выигрыша, решает одно: часто ли угадывает черновая модель.

Авторы заявляют ускорение до 6.5 раза. Я повторил их метод на Qwen3-1.7B и бесплатной Kaggle T4: на математике и коде получилось около двух с половиной раз, на русскоязычных запросах — около 0.95, то есть генерация стала медленнее обычной. Последнее, как выяснится, свойство не языка: черновая модель угадывает здесь редко, и вдобавок ей дают угадывать слишком далеко вперёд. Если сократить, насколько далеко она заглядывает, те же запросы выходят в небольшой плюс. Оба числа следуют из одного выражения для ускорения. В §1 я его вывожу, дальше измеряю по отдельности всё, что в него входит, и показываю, что когда перевешивает.

Run All работает без прикреплённых данных: нужны GPU T4 и включённый интернет; прогон занимает около 35 минут. English summary is at the bottom of §14.

Что сделано

- Запустил EAGLE-3 на Qwen3-1.7B кодом официального репозитория — [§4](#).
- Сравнил обычную авторегрессионную генерацию и EAGLE-3 на пяти наборах запросов — [§5](#).
- Построил, разобрал и измерил дерево черновиков — [§6–§8](#).
- Прогнал тот же протокол на цели вдвое крупнее, Qwen3-4B — [§11](#).

Что вы узнаете

1. Почему проверка целого дерева черновиков стоит столько же, сколько генерация одного токена — по замеру, а не по спецификации карты.
2. Сколько на самом деле стоит обычный шаг генерации: вчетверо больше теоретического минимума, и часть выигрыша метода — это амортизация неоптимальности базового варианта.
3. Как выглядит дерево черновиков изнутри и сколько добавляет ветвление против простой цепочки.
4. Какая форма дерева оптимальна — по одному параметру за раз и с повторами, а не «покрутили три ручки сразу».
5. Как убедиться, что метод не портит выдачу: потоленно при жадном декодировании и статистически при сэмплинговании.
6. Когда метод вредит: на чужом для черновой модели домене и глубоком дереве он проигрывает обычной генерации.
7. Окупается ли метод лучше на большей цели: распространённую интуицию я довожу до замера на Qwen3-4B.

Содержание

- [0. Параметры прогона](#)
- [1. Арифметика декодирования](#)
- [2. KV-кэш: цена памяти и почему дерево вообще возможно](#)
- [3. Что именно меняет EAGLE-3 — и что из этого видно в коде](#)
- [4. Запуск: три способа генерации на одной модели](#)
- [5. Бенчмарк на наборах из статьи](#)
- [6. Дерево черновиков: анатомия одного шага](#)
- [7. Форма дерева: два среза по одному параметру](#)
- [8. Куда уходит время цикла](#)
- [9. Температура: где проверяется главная гарантия метода](#)
- [10. Совпадение с обычной генерацией при \$T = 0\$](#)
- [11. Тот же метод, цель вдвое крупнее](#)
- [12. Что не сработало](#)
- [13. Выводы](#)
- [14. Прогнать у себя, что читать дальше, лицензии](#)

0. Параметры прогона

Четыре строки ниже — всё, что нужно поменять при форке, чтобы проверить метод в другом сочетании условий. Остальное соберётся под них само: загрузятся веса, а наборы запросов, таблицы и графики построятся по тем же настройкам.

- `BASE_MODEL` — целевая модель, генерацию которой ускоряем.
- `EA_MODEL` — черновая голова: та самая маленькая модель из вступления, обученная именно под эту цель. Список готовых голов — в README репозитория EAGLE, ссылка в §14.
- `QUICK` — укороченный прогон примерно на 10 минут вместо 35: меньше вопросов в наборах, меньше повторов в замерах, а раздел про масштабирование (§11) пропускается целиком — он требует загрузки второй пары моделей.
- `LANG_SET` — имя набора, на котором проверяется работа вне домена черновой головы. На нём я и смотрю, что бывает, когда голова обучена не на тех данных.

```
BASE_MODEL = "Qwen/Qwen3-1.7B"  
EA_MODEL   = "AngelSlim/Qwen3-1.7B_eagle3"  
QUICK      = False  
LANG_SET   = "Русский"
```

Ячейка ниже готовит окружение: ставит нужную версию `transformers`, забирает репозиторий EAGLE на зафиксированном коммите и проверяет, что выдали именно ту карту, на которой всё это считалось. Заодно печатает версии — базовый образ Kaggle обновляется и молча ломает старые ноутбуки.

```
GPU: Tesla T4 (sm_75)  
torch 2.10.0+cu128 | transformers 4.53.1  
EAGLE импортирован, коммит cb7e0841fe0c  
python 3.12.13 | numpy 2.0.2 | pandas 2.3.3
```

1. Арифметика декодирования

Авторегрессионная генерация выдаёт один токен за проход. На каждом шаге GPU обязан прочитать все веса модели ради одного токена. На каждый прочитанный байт приходится около одной операции, а карта рассчитана примерно на 200:

$$\begin{aligned} \mathcal{L}_{\text{decode}} &= \frac{2 \cdot \text{op/param}}{\text{byte/param}} = 1, \quad \\ \mathcal{L}_{\text{T4}} &= \frac{65 \cdot \text{TFLOPS}}{320 \cdot \text{GB/s}} \approx 200 \end{aligned}$$

Слева арифметическая интенсивность декодирования в fp16, справа паспортный баланс T4. Это в 200 раз ниже точки равновесия: шаг декодирования стоит столько, сколько занимает

прокачка весов через шину памяти. Вычислители при этом простаивают. Об этом лекция «Deep Learning Arithmetic»: генерация упирается в память, а не в вычисления.

Если время шага определяется чтением весов, то обработка не одного токена, а сразу нескольких почти не меняет стоимость: веса читаются один раз в обоих случаях. Проверка k черновиков должна быть так же дешева, как генерация одного токена.

Обычно это утверждение цитируют. Я его измерил и заодно нашёл границу, где «почти бесплатно» кончается.

Загружаю целевую модель и черновую голову. Обе весят немного: целевая — 3.2 ГБ в половинной точности, голова — ещё 0.27 ГБ, так что всё вместе помещается в 16 ГБ T4.

загружено за 35 с

целевая модель: 1.72B параметров, 28 слоёв, 3.20 ГБ в fp16

черновая голова: 137M (8.0% от целевой модели)

Первый замер: сколько времени занимает один проход целевой модели, если подать ей не один токен, а сразу несколько. Если рассуждение выше верно, кривая должна быть плоской — веса читаются один раз независимо от того, сколько токенов обрабатывается.

токенов	мс	мс/токен	к n=1
1	43.5	43.523	1.00x
2	46.1	23.041	1.06x
4	44.7	11.182	1.03x
8	47.5	5.941	1.09x
16	46.9	2.933	1.08x
24	47.0	1.957	1.08x
32	47.2	1.475	1.08x
48	46.9	0.977	1.08x
64	47.3	0.739	1.09x
96	48.7	0.507	1.12x
128	47.4	0.370	1.09x
192	49.0	0.255	1.13x
256	61.0	0.238	1.40x
384	84.3	0.219	1.94x
512	120.5	0.235	2.77x

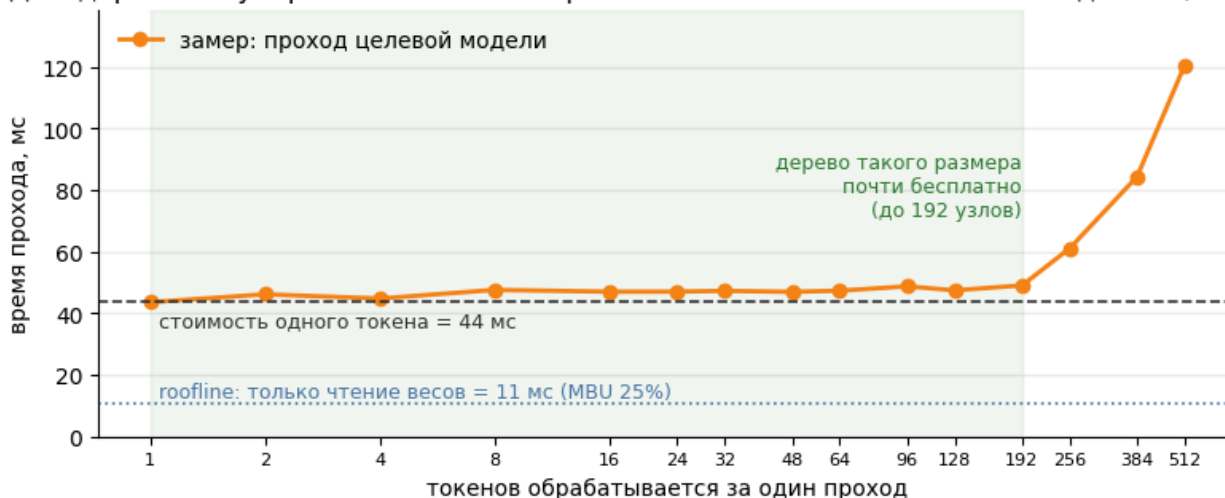
один токен: 43.5 мс

roofline-предсказание (только чтение весов): 10.8 мс

=> утилизация полосы памяти (MBU): 25%

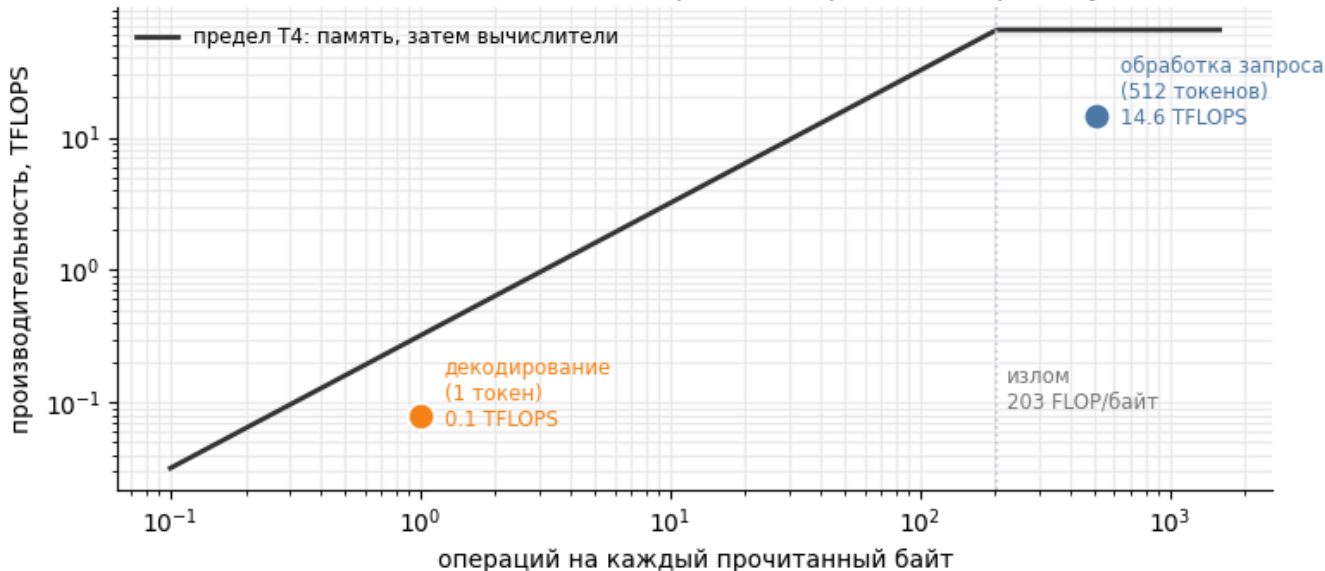
=> проход дорожает меньше чем на 25% вплоть до 192 токенов

Декодирование упирается в память: обработать 100 токенов почти так же дёшево, как один



Вывод. Кривая плоская почти до 192 токенов: обработать 100 токенов стоит почти столько же, сколько один. На этом запасе и работает спекулятивное декодирование. Второе наблюдение — расстояние до пунктирной линии: обычный шаг вчетверо дороже теоретического минимума, то есть запас есть и у самой обычной генерации, и часть будущего ускорения даст именно он.

Где мы живём на goofline: декодирование прижато к пределу памяти



декодирование (1 токен)	операций на байт	1.0		0.1 TFLOPS из 65
обработка запроса (512 токенов)	операций на байт	512.0		14.6 TFLOPS из 65

Вывод. Декодирование одного токена живёт в левой части диаграммы — там, где производительность ограничена шиной памяти, а вычислители простаивают. Обработка запроса на 512 токенах сдвигается вправо на два порядка и упирается уже в другой предел. Спекулятивное декодирование двигает шаг генерации из

левой зоны правее: за те же прочитанные байты выполняется больше полезной работы.

Формула, к которой сводится весь разбор

Длина принятия τ — среднее число токенов, которые цикл выдаёт за один проход целевой модели: принятые черновики плюс бонусный токен, то есть в коде это `accept_length + 1`. У обычной генерации $\tau = 1$ по определению: один проход, один токен.

Тогда выигрыш описывается так:

$$S \approx \frac{\tau \cdot t_{\text{step}}}{t_{\text{verify}} + (d+1) \cdot t_{\text{draft}}}$$

Обозначения:

- S — ускорение относительно обычной генерации;
- τ — длина принятия;
- d — глубина дерева;
- t_{step} — время обычного шага генерации;
- t_{verify} — время проверки всего дерева за один проход целевой модели;
- t_{draft} — время одного шага черновой модели.

В числителе — сколько токенов приносит цикл, в знаменателе — во что он обходится. Дальше я измеряю по отдельности каждую величину: t_{verify} здесь и в §8, t_{draft} в §7 и §8, τ в §5. Все результаты разбора складываются в эту формулу, включая тот, где ускорение оказывается меньше единицы.

Базовый вариант. Замеренный шаг примерно вчетверо дороже *roofline*-предсказания: шина памяти загружена на четверть. Разница — это накладные расходы реализации: attention без flash-ядер, пооперационный запуск CUDA-ядер из Python, отсутствие CUDA-графов. Спекулятивное декодирование делит эти накладные расходы на τ принятых токенов. Поэтому *часть* ускорения, которое я дальше измеряю, объясняется не выигрышем по памяти, а амортизацией неоптимальности базового варианта. На вылизанном стеке (vLLM, SGLang, CUDA-графы) обычный шаг ближе к пределу памяти, и ускорение от спекуляции получается скромнее: независимые замеры на vLLM дают 1.3–2× там, где авторские фреймворки показывают 4–6× ([«Performance or Illusion?», 2601.11580] (<https://arxiv.org/abs/2601.11580>)).

Второе ограничение — `batch = 1`: я измеряю латентность. В проде важнее пропускная способность, а там картина другая: при большом батче вычислители

загружены и без всякой спекуляции, проверять черновики уже нечем, и выигрыш схлопывается. В самой статье это видно: 4–6× при batch = 1 против 1.38× в SGLang при batch = 64. Спекулятивное декодирование лечит decode и только его — обработка запроса живёт на roofoffline правее и в лечении не нуждается.

2. KV-кэш: цена памяти и почему дерево вообще ВОЗМОЖНО

Замер выше показал, что дерево проверяется почти даром. Осталось понять, где это дерево живёт между шагами и почему вообще может там жить.

Без кэша внимания генерация была бы квадратичной: каждый новый токен пересчитывал бы ключи и значения для всего префикса. KV-кэш хранит их и превращает шаг в линейный по длине контекста. Цена — память, и она считается точно.

Для нашей модели: 28 слоёв, 8 KV-голов против 16 голов внимания (GQA: вдвое экономнее обычного внимания), размер головы 128, 2 тензора (K и V), 2 байта на число.

Спекулятивное декодирование добавляет к этому требование, которого нет у обычной генерации. За один проход проверяется целое дерево, но принимается только один путь в нём. Значит, кэш должен уметь три вещи: пускать каждый узел смотреть только на своих предков (**tree attention** — маска, а не обычная каузальная), хранить K и V для всех узлов дерева сразу, а после проверки оставить только выигравший путь, выкинув остальные ветки без пересчёта.

В EAGLE это сделано без аллокаций в цикле: буфер под весь контекст выделяется один раз (`initialize_past_key_values`), а когда кандидат принят, его позиции копируются на своё место внутри того же буфера (`dst.copy_(tgt)` в `update_inference_inputs`), после чего указатель длины сдвигается. Корректность здесь держится на том, что K и V каждого узла считались с учётом только его предков — а значит, для принятого пути они ровно те же, какими были бы при обычном последовательном декодировании. Отброшенные ветки не оставляют следа.

Считаю, во что обходится кэш на этой модели и сколько занимает в нём дерево черновиков.

слоёв 28 | KV-голов 8 (голов внимания 16, то есть GQA x2) | head_dim 128

KV на один токен: 112 КБ

контекст 512 → 56 МБ (1.7% от весов модели)

контекст 2048 → 224 МБ (6.8% от весов модели)

контекст 8192 → 896 МБ (27.3% от весов модели)

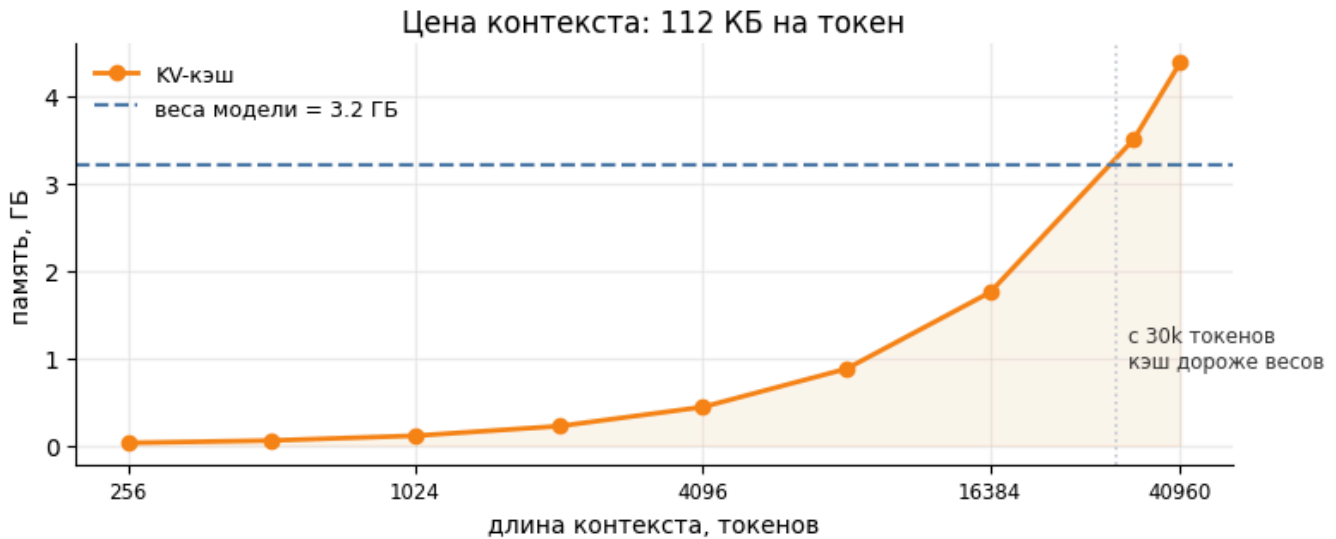
контекст 40960 → 4480 МБ (136.5% от весов модели)

дерево из 32 узлов держит в кэше 3.5 МБ — меньше 0.11% от весов

дерево из 96 узлов держит в кэше 10.5 МБ — меньше 0.32% от весов

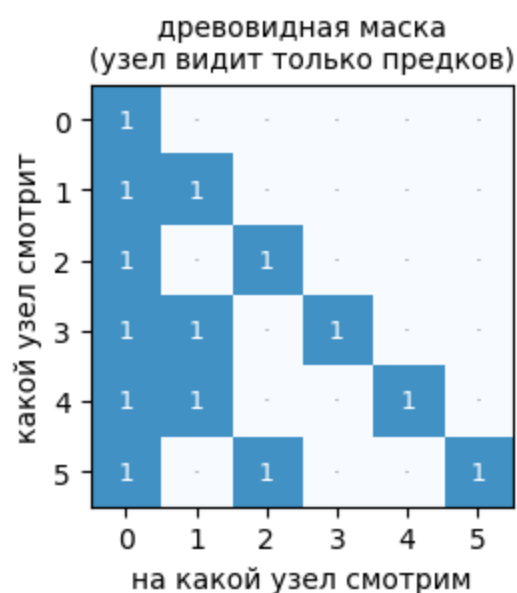
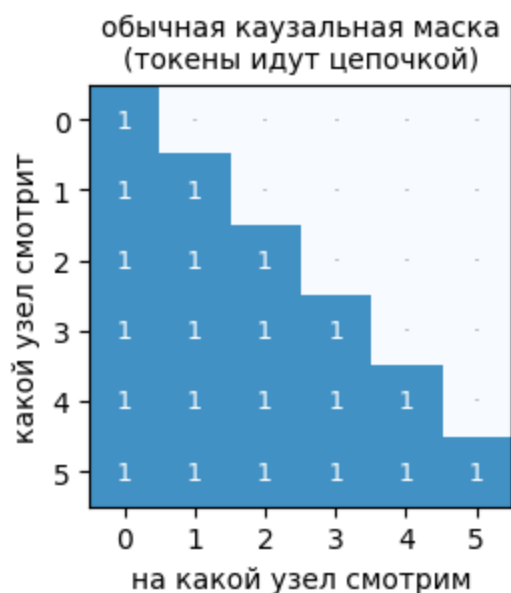
дерево из 256 узлов держит в кэше 28.0 МБ — меньше 0.85% от весов

веса модели в fp16: 3.20 ГБ | занято на GPU сейчас: 4.07 ГБ



Вывод. На коротком контексте KV-кэш почти незаметен, но растёт он линейно и на предельных для Qwen3 40 тысячах токенов обгоняет сами веса модели.

Дерево черновиков на этом фоне бесплатно: сотня узлов занимает единицы мегабайт. Именно поэтому спекулятивное декодирование ничего не стоит по памяти — оно расходует то, чего в избытке, ради того, чего не хватает. И обратная сторона: на длинном контексте арифметика §1 меняется, потому что шаг тащит уже и веса, и кэш, поэтому дорожают одновременно и обычная генерация, и проверка дерева. Все замеры этого разбора сняты на коротких запросах, где кэш ещё ничего не решает.



Дерево: $1 \leftarrow 0$ $2 \leftarrow 0$ $3 \leftarrow 1$ $4 \leftarrow 1$ $5 \leftarrow 2$

Узлы 3 и 5 лежат в разных ветках и в древовидной маске друг друга не видят, хотя в обычной каузальной маске узел 5 видел бы узел 3 просто потому, что тот раньше.

Вывод. Каузальная маска разрешает каждому токenu смотреть на всё, что левее. Для цепочки это верно. В дереве так нельзя: узлы из разных веток лежат в одном тензоре, но представляют взаимоисключающие продолжения. Древовидная маска оставляет каждому узлу только его предков; без неё ветки перепутались бы, а проверка потеряла бы смысл.

3. Что именно меняет EAGLE-3 — и что из этого видно в коде

Кэш умеет хранить дерево целиком и оставлять от него принятый путь. Теперь — чем третья версия метода отличается от первых двух и что из этого видно не в статье, а в коде.

EAGLE-3 — третья работа серии: EAGLE-1 (ICML'24), EAGLE-2 (EMNLP'24), EAGLE-3 (NeurIPS'25). Заявленное в ней ускорение до 6.5 раза — лучшая ячейка таблицы (HumanEval на Vicuna-13B); средние по моделям скромнее, 5.51 на Vicuna-13B и 4.44 на LLaMA-3.1-8B, и всё это при жадном декодировании, $\text{batch} = 1$ и в собственном фреймворке авторов. Там же есть вторая цифра, и измеряет она другое: в SGLang при $\text{batch} = 64$ прирост пропускной способности составляет 1.38 раза.

EAGLE-1 ([2401.15077](https://arxiv.org/abs/2401.15077)) заметил, что предсказывать следующий токен маленькой моделью тяжело, а предсказывать **признак** — скрытое состояние верхнего слоя целевой модели — заметно легче: признаки регулярнее токенов. Черновая модель там устроена как один трансформерный слой, который получает на вход признак `f_t` и эмбединг уже известного токена `x_{t+1}`, а затем авторегрессивно продолжает цепочку признаков. Второй вход

снимает неопределённость сэмплирования: черновая модель знает, какой токен на самом деле выпал.

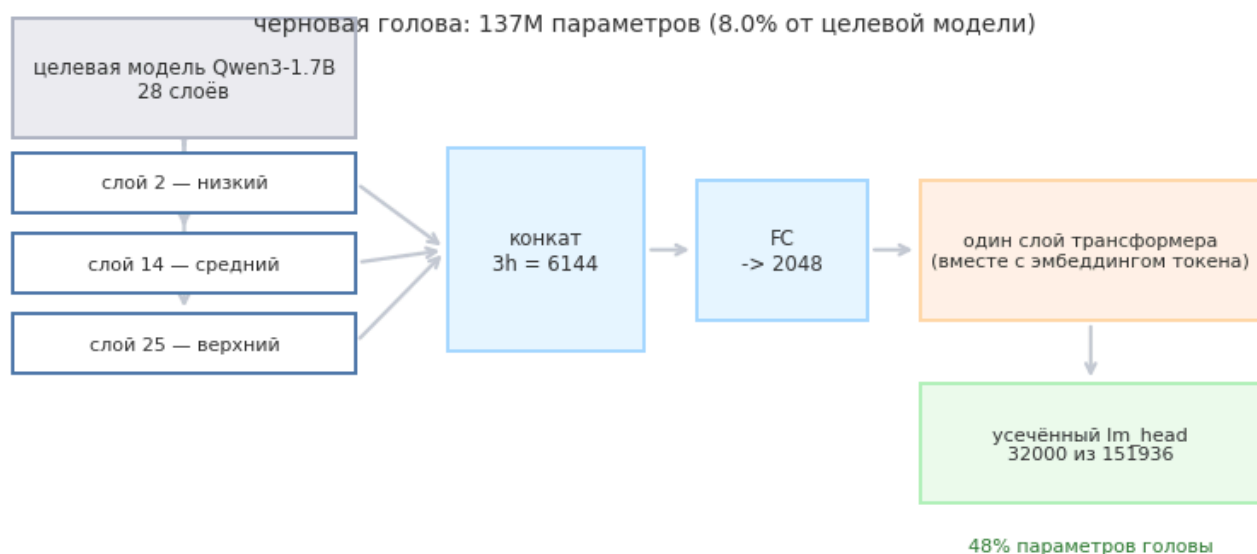
EAGLE-2 ([2406.16858](#)) сделал дерево черновиков динамическим. Раньше форма дерева была фиксированной; теперь узлы раскрываются по уверенности черновой модели, и в дерево отбираются `total_token` лучших по кумулятивной логарифмической вероятности. Там, где черновая модель уверена, дерево растёт вглубь; где сомневается — вширь.

EAGLE-3 ([2503.01840](#)) вносит три изменения:

1. **Черновая модель перестаёт предсказывать признак.** Раньше в лоссе был член `l_fea` — регрессия на признак целевой модели. Он работал как регуляризатор и упирался в потолок: качество черновой модели переставало расти с объёмом обучающих данных. EAGLE-3 обучает черновую модель только предсказывать токены и получает главный результат статьи: длина принятия начинает расти с объёмом данных. Тонкость: авторегрессия по признакам никуда не делась, черновая модель по-прежнему передаёт своё скрытое состояние на следующий шаг. Ушёл именно лосс, который заставлял это состояние совпадать с признаком целевой модели.
2. **Слияние трёх уровней.** Вместо одного признака с верхнего слоя — конкатенация скрытых состояний с низкого, среднего и верхнего слоёв, сжатая линейным слоем $3h \rightarrow h$. Верхний признак заточен под ближайший токен и теряет информацию, полезную на несколько шагов вперёд.
3. **Training-time test.** При генерации черновая модель идёт по цепочке собственных выходов, которых она при обучении не видела, и накапливает дрейф. EAGLE-3 при обучении раскатывает черновую модель на её собственных предсказаниях, имитируя инференс. Ближайший предок идеи — HASS ([2408.15766](#)).

Три подробности, которых в статье нет, а в загруженных весах видны сразу: с каких именно слоёв берутся признаки, насколько урезан словарь черновой модели и какой параметр остался в коде нерабочим.

- 1) Слои целевой модели, с которых берётся слияние (в коде: `idx == 2, L//2, L-3`):
 низкий 2 | средний 14 | верхний 25 из 28 слоёв
`fc: (2048, 6144)` – сжимает $3h \rightarrow h$
- 2) Усечённый словарь черновой модели – не деталь, а условие применимости:
 словарь целевой модели 151936, словарь черновой модели 32000
`lm_head` черновой модели: 66М параметров = 48% всей головы
 с полным словарём голова весила бы 382М = 22% от целевой модели (вместо 8.0%)
 то есть на таком маленьком целевой модели EAGLE-3 без усечения словаря не окупилась бы
- 3) Как связаны словари (`d2t` хранит СМЕЩЕНИЯ, а не индексы):
`d2t: (32000,) torch.int64`, применяется как `token = i + d2t[i]`
`t2d: (151936,) torch.bool` – маска покрытия словаря целевой модели
 покрыто токенов целевой модели: 32000 из 151936
 токен вне `draft`-словаря черновая модель предложить не может, но целевая модель его свободно выдаёт бонусным токеном – на корректность это не влияет
- 4) Параметр `threshold`: живёт в конструкторе, но в EAGLE-3-ветке не используется.
`self.threshold` задан в `__init__`, упоминаний в `topK_genrate: 0`
 (в `cnets1.py` – черновая модель EAGLE-1/2 – он ещё работал как порог отсечения)



слияние трёх уровней — главное отличие EAGLE-3 от EAGLE-2

Вывод. Черновая модель устроена куда проще целевой модели: три отвода скрытых состояний, линейное сжатие, один слой трансформера и выходной словарный слой. Почти половину всех её параметров занимает как раз этот слой, и он усечён до 32 тысяч токенов вместо полного словаря: без урезания черновая модель весила бы пятую часть целевой и на ней бы не окупилась.

4. Запуск: три способа генерации на одной модели

Сначала про код и веса. Код я беру из официального репозитория [SafeAILab/EAGLE](#) на коммите `cb7e0841` — оттуда же `eagenerate`, `naivegenerate`, построение дерева и наборы вопросов. Черновую голову беру ту, которую этот же репозиторий указывает в своей таблице весов для Qwen3-1.7B: [AngelSlim/Qwen3-1.7B_eagle3](#). В таблице она помечена как неавторская — обучила её команда [AngelSlim](#) в Tencent: у самих авторов EAGLE готовые головы есть только под Vicuna и LLaMA. Для чтения цифр это важнее, чем кажется: голова обучена на англоязычных диалогах и вне своего домена ведёт себя совсем иначе (§5).

Все три работают с одними и теми же весами: модель одна, различается только цикл, который её вызывает.

generate из transformers — то, что получает обычный пользователь. Это честный внешний ориентир, но у него другой бэкэнд внимания и другой кэш, поэтому он не изолирует эффект спекуляции.

naivegenerate — обычный цикл генерации из репозитория EAGLE (в статьях его называют *vanilla autoregressive*): та же модифицированная модель, тот же статический KV-буфер, что и у спекулятивного цикла. Относительно него статья и считает свои ускорения, и он же изолирует вклад метода: между ним и `eagenerate` одно отличие — спекуляция.

eagenerate — собственно EAGLE-3.

Дальше все цифры я привожу к обоим базовым вариантам сразу. Неприятная сторона:

`naivegenerate` на нашей карте оказывается медленнее штатного `generate` примерно на 10% — он не использует flash-attention. Значит, ускорение к нему слегка льстит методу. Поэтому в итоговой таблице стоят обе колонки.

Все три останавливаются по EOS, а тексты у них расходятся (об этом §10), поэтому скорость в токенах в секунду считалась бы на разной работе. Чтобы сравнение было честным, в замерах я подавляю конец последовательности и заставляю каждый способ выдать одинаковое число токенов.

Ниже — общая обвязка замеров: единый способ собрать запрос, секундомер вокруг генерации и переключатель формы дерева. Там же живёт подавление конца последовательности, о котором сказано абзацем выше.

5. Бенчмарк на наборах из статьи

Три способа готовы, нужен честный набор задач. Самодельные запросы — слабое место любого замера скорости: их всегда можно подобрать под удобный ответ. К счастью, репозиторий EAGLE везёт с собой ровно те наборы, на которых считает статью: MT-Bench (диалог), GSM8K (арифметические задачи), HumanEval (код), Alpaca (инструкции) — по 80

вопросов каждый. Беру из каждого первые пять: полный прогон по 320 вопросам в бесплатную сессию не влезает.

Пятый набор я написал сам: пять русскоязычных запросов, никакого публичного бенчмарка. Черновая голова обучена сообществом на англоязычных данных, и мне интересен режим, в котором метод работает вне своего домена. Как увидим, он и даёт единственный отрицательный результат разбора.

Порядок вызовов внутри вопроса фиксирован — сначала штатный `generate`, потом `naivegenerate`, потом `eagenerate`; рандомизации нет. Все три запускаются подряд на одном вопросе, поэтому нагрев карты действует на них почти одинаково: разброс обычной генерации между повторами составляет доли процента (§7 печатает его явно).

Числа этого раздела сняты на стартовой форме дерева `60/7/10` (`total_token / depth / top_k`), заданной при загрузке модели. В §7 выяснится, что эта форма не оптимальна: более мелкое дерево заметно выигрывает. §7 печатает и срез, и победившую форму сам, а точная цифра гуляет между прогонами. Полный бенчмарк на выбранной форме я не прогонял: сессия Kaggle не резиновая, а вывод раздела — про разрыв между доменами в два с половиной раза — это отношение двух ускорений, снятых на одной форме, так что общий сдвиг из него сокращается.

Собираю пять наборов: четыре из репозитория статьи и свой русскоязычный.

```
MT-Bench : 5 вопросов | первый: 'Compose an engaging travel blog post about a recent trip to Hawa'
GSM8K     : 5 вопросов | первый: "Darrell and Allen's ages are in the ratio of 7:11. If their tota"
HumanEval : 5 вопросов | первый: 'Complete the code I provided.\n\ndef below_threshol
d(l: list, t: i'
Alpaca     : 5 вопросов | первый: 'What would be the best type of exercise for a perso
n who has art'
Русский    : 5 вопросов | первый: 'Объясни в трёх предложениях, почему небо голубое.'
```

Отсюда берутся все числа §5: каждый способ выдаёт фиксированное число токенов, и время делится на них.

MT-Bench	q0: naive	46.4 мс/ток		eagle	26.8 мс/ток		τ 2.55
MT-Bench	q1: naive	46.5 мс/ток		eagle	18.2 мс/ток		τ 3.78
MT-Bench	q2: naive	46.1 мс/ток		eagle	23.5 мс/ток		τ 2.92
MT-Bench	q3: naive	46.3 мс/ток		eagle	24.6 мс/ток		τ 2.74
MT-Bench	q4: naive	46.0 мс/ток		eagle	26.9 мс/ток		τ 2.54
GSM8K	q0: naive	46.4 мс/ток		eagle	18.4 мс/ток		τ 3.75
GSM8K	q1: naive	46.1 мс/ток		eagle	20.9 мс/ток		τ 3.25
GSM8K	q2: naive	46.3 мс/ток		eagle	15.4 мс/ток		τ 4.43
GSM8K	q3: naive	46.8 мс/ток		eagle	20.8 мс/ток		τ 3.27
GSM8K	q4: naive	46.7 мс/ток		eagle	18.6 мс/ток		τ 3.68
HumanEval	q0: naive	45.8 мс/ток		eagle	15.9 мс/ток		τ 4.29
HumanEval	q1: naive	45.4 мс/ток		eagle	19.3 мс/ток		τ 3.50
HumanEval	q2: naive	45.7 мс/ток		eagle	20.4 мс/ток		τ 3.33
HumanEval	q3: naive	46.4 мс/ток		eagle	17.5 мс/ток		τ 4.02
HumanEval	q4: naive	47.2 мс/ток		eagle	17.7 мс/ток		τ 3.98
Alpaca	q0: naive	46.3 мс/ток		eagle	25.4 мс/ток		τ 2.71
Alpaca	q1: naive	46.1 мс/ток		eagle	21.7 мс/ток		τ 3.11
Alpaca	q2: naive	46.2 мс/ток		eagle	18.8 мс/ток		τ 3.64
Alpaca	q3: naive	45.4 мс/ток		eagle	18.5 мс/ток		τ 3.71
Alpaca	q4: naive	46.4 мс/ток		eagle	24.4 мс/ток		τ 2.77
Русский	q0: naive	46.0 мс/ток		eagle	59.6 мс/ток		τ 1.12
Русский	q1: naive	47.1 мс/ток		eagle	39.0 мс/ток		τ 1.74
Русский	q2: naive	46.3 мс/ток		eagle	57.8 мс/ток		τ 1.17
Русский	q3: naive	46.5 мс/ток		eagle	41.4 мс/ток		τ 1.64
Русский	q4: naive	45.8 мс/ток		eagle	53.1 мс/ток		τ 1.25

Свожу замеры по наборам. Разброс между вопросами внутри набора показываю рядом со средним: без него непонятно, отличается ли один набор от другого или это шум.

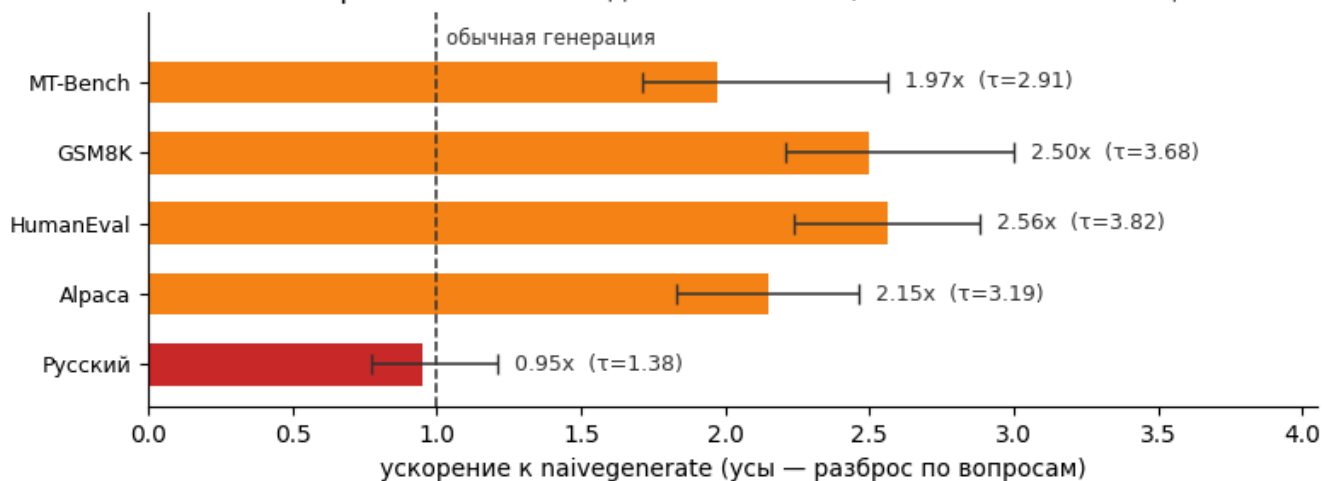
	tau	tau_min	tau_max	speedup	sp_min	sp_max	speedup_hf
bench							
MT-Bench	2.910000	2.540000	3.780000	1.970000	1.710000	2.560000	1.770000
GSM8K	3.680000	3.250000	4.430000	2.500000	2.210000	3.000000	2.220000
HumanEval	3.820000	3.330000	4.290000	2.560000	2.240000	2.880000	2.270000
Alpaca	3.190000	2.710000	3.710000	2.150000	1.830000	2.460000	1.940000
Русский	1.380000	1.120000	1.740000	0.950000	0.770000	1.210000	0.850000

Итого по всем наборам: τ = 3.00, ускорение 2.03x к naivegenerate, 1.81x к штатному generate

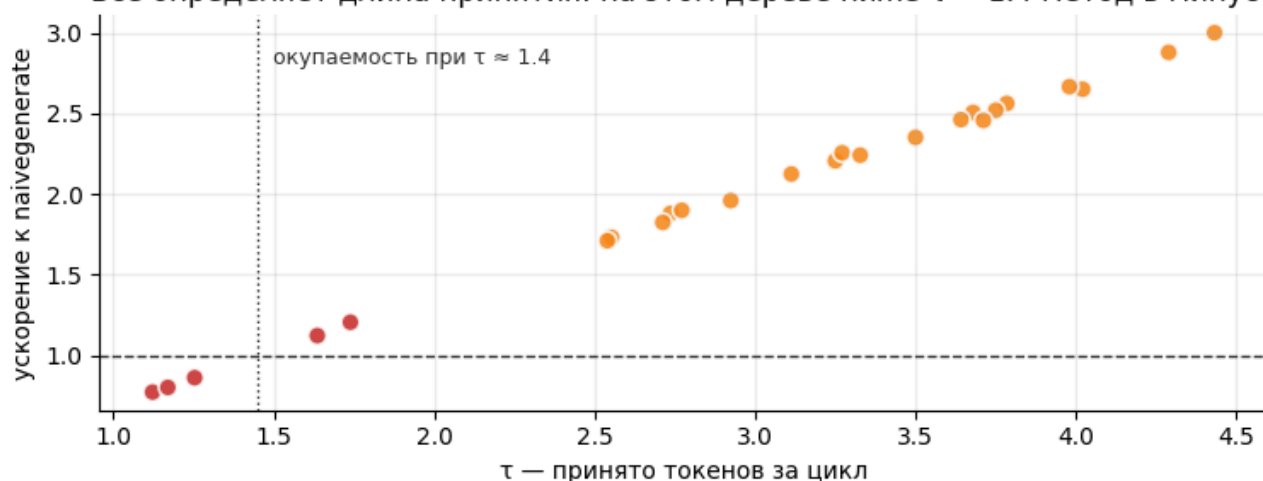
английские наборы: τ = 3.40, ускорение 2.30x

русскоязычный набор: τ = 1.38, ускорение 0.95x

Ускорение зависит от домена сильнее, чем от чего-либо ещё



Всё определяет длина принятия: на этом дереве ниже $\tau \approx 1.4$ метод в минусе



Вывод. Домен решает всё. На англоязычных наборах длина принятия держится около 3.4, и метод ускоряет вдвое с лишним; на русскоязычных она падает примерно до 1.4, и на этой — стартовой, глубокой — форме дерева та же машинерия начинает проигрывать обычной генерации. Разброс между вопросами внутри набора заметно меньше разрыва между наборами — значит, дело в домене, а не в отдельных неудачных запросах. Правый график показывает механику: ускорение почти линейно по длине принятия, и ниже точки окупаемости, отмеченной на графике, кривая уходит под единицу.

Что это значит на практике. На англоязычных наборах метод даёт стабильное ускорение, на русскоязычных — *замедляет* генерацию. Этот набор одновременно и вне домена черновой головы, и на другом языке — разделить два обстоятельства этот прогон не может. Голову обучало сообщество на англоязычных диалогах, так что данные обучения — причина вероятная. Проверить её мог бы англоязычный набор вне домена или голова, обученная на русскоязычных данных; ни того ни

другого здесь нет. Так или иначе, вне своего домена она угадывает редко, т падает почти до единицы, и цикл «построить дерево → проверить → отбросить» вырождается в чистые накладные расходы поверх обычного шага.

Только замер снят на *стартовой* форме дерева, а она глубокая. На форме, которую выбирает §7, замедления уже нет: §11 прогоняет подмножество тех же запросов и получает небольшой плюс. Значит, замедление — свойство не одного лишь домена, а домена и глубины вместе. Тот же срез по глубине §7 снимает и на русскоязычном наборе: кривая там пересекает единицу.

Спекулятивное декодирование гарантирует качество, но не гарантирует выигрыш: длину принятия надо измерять на своём распределении запросов, а не брать из статьи.

6. Дерево черновиков: анатомия одного шага

Посмотрю на одно дерево целиком.

Форма дерева задаётся тремя параметрами:

- `top_k` задаёт ширину: сколько продолжений раскрывается у каждого узла;
- `depth` задаёт, сколько шагов расширения идёт после первого прохода черновой модели: цикл стоит `depth + 1` запусков, и в дереве `depth + 1` уровней;
- `total_token` задаёт бюджет: сколько узлов из всех раскрытых остаётся в дереве.

Строится оно так. Черновая модель делает один проход, потом ещё `depth` шагов; на каждом шаге у каждого из `top-k` текущих узлов раскрываются свои `top-k` продолжений, и получается пул кандидатов. Каждому приписана кумулятивная логарифмическая вероятность — сумма по всему пути от корня. Из пула отбираются `total_token` лучших по этой сумме, и уже по ним собираются древовидная маска и позиционные индексы. Динамический отбор — вклад EAGLE-2: у EAGLE-1 форма дерева задавалась заранее и не зависела от контекста.

Целевая модель проверяет всё дерево за один проход с древовидной маской из §2. Из её логитов извлекаются все корневые пути; при жадном декодировании путь принимается ровно до той позиции, где очередной черновик перестал совпадать с `argmax` целевой модели. К принятому пути добавляется **бонусный токен** — тот, который целевая модель предсказала сама на последней принятой позиции. Поэтому даже при нулевом принятии цикл выдаёт один токен, как обычная генерация, и метод не может быть хуже по числу проходов целевой модели.

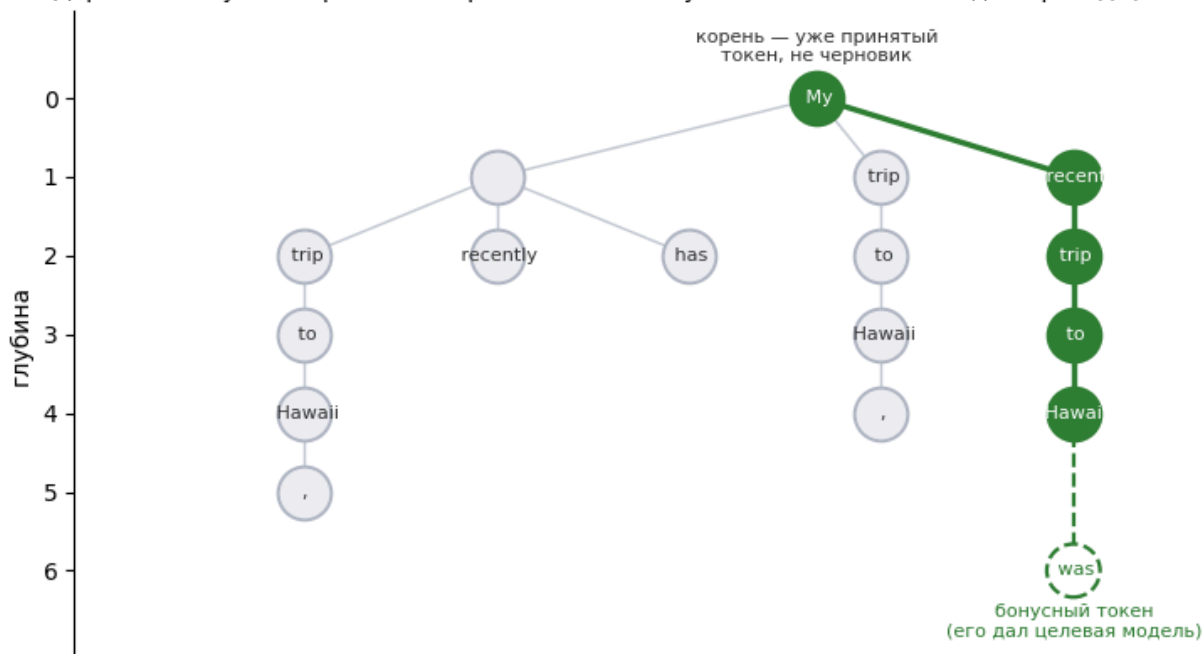
Ниже — инструментированная копия цикла: та же логика, но с записью дерева и результата проверки на каждом шаге.

Чтобы увидеть дерево, повторяю цикл генерации построчно и на каждом шаге сохраняю его состав, древовидную маску и то, какой путь прошёл проверку.

шагов 43, выдано 98 токенов, $\tau = 2.28$

приято за цикл: [2, 1, 3, 3, 2, 2, 1, 3, 1, 4, 2, 1, 3, 2, 1, 2, 1, 3, 1, 2, 2, 4, 3, 2, 2, 2, 5, 2, 2, 1, 3, 1, 5, 2, 1, 1, 3, 3, 5, 1, 4, 2, 2]

Дерево из 16 узлов: принято 4 черновиков + 1 бонусный = 5 токенов за один проход целевой модели



Пути этого дерева (первые 6):

[отвергнут] My → → recently

[отвергнут] $My \rightarrow \rightarrow has$

[ПРИНЯТ] My -> recent -> trip -> to -> Hawaii

[отвергнут] My -> trip -> to -> Hawaii -> ,

[отвергнут] My -> -> trip -> to -> Hawaii -> ,

Вывод. Видно, чем дерево отличается от цепочки: черновая модель предложила несколько взаимоисключающих продолжений, а целевая проверила их все за один проход. Зелёный путь совпал с её собственным выбором и принят целиком, остальные ветки отброшены. Последний токен пути даёт сама целевая модель — поэтому даже при полном промахе цикл не хуже обычного шага.

Что добавляет ветвление

Смотрю на две вещи по отдельности.

Первая — частота принятия по глубине: какая доля циклов дожила до глубины 1, 2, 3 и так далее. Она показывает, где именно черновая модель выдыхается.

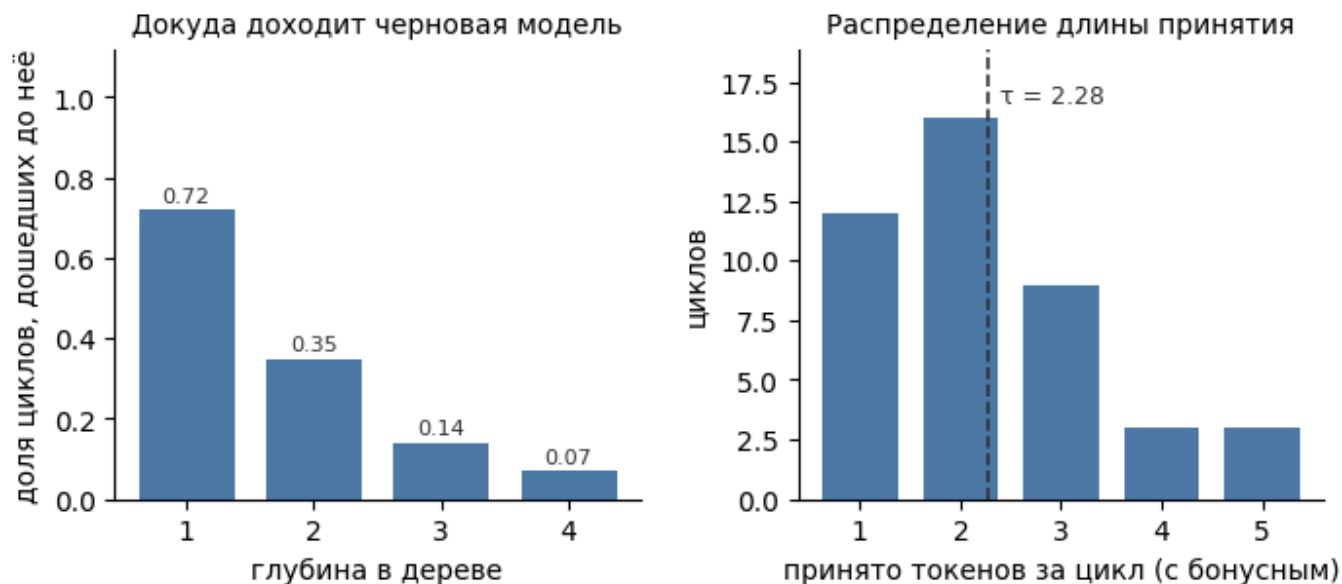
Вторая — сколько добавляет само ветвление. Вырожденный случай дерева при `top_k = 1` — это классическое спекулятивное декодирование цепочкой: черновая модель предлагает одну

последовательность, целевая модель её проверяет. Сравниваю цепочку и дерево той же глубины на одном запросе, при одинаковом числе проходов целевой модели.

τ дерева (16/5/3): 2.28 токенов за цикл

τ цепочки (6/5/1): 1.64 — та же черновая модель, та же глубина, без ветвления

=> ветвление добавляет 39% к длине принятия при том же числе проходов целевой модели



Вывод. Частота принятия падает с глубиной быстро: хотя бы один черновик принимают 72% циклов, до второго уровня доходят 35%, до четвёртого — 7%, и четвёртый здесь предел. Замер в следующем разделе это подтверждает, а вывод виден уже здесь: наращивать глубину дальше нескольких уровней бессмысленно, потому что платить за неё приходится всегда, а окупается она редко.

7. Форма дерева: два среза по одному параметру

Из §1 следует несимметричное ожидание. **Ширина** дерева — это лишние узлы в том же самом проходе проверки, а он у нас плоский примерно до 192 токенов, то есть ширина почти бесплатна. **Глубина** — это дополнительные последовательные запуски черновой модели, каждый со своей фиксированной ценой, которую ничем не скрыть.

Чтобы это проверить, менять надо по одному параметру за раз. Сначала фиксирую `total_token` и двигаю глубину, потом фиксирую глубину и двигаю `total_token`. Каждая точка измеряется трижды, на графике — среднее и разброс: без этого выбирать конфигурацию по разнице в пару процентов бессмысленно, потому что шум карты между повторами того же порядка.

Оба среза считает ячейка ниже; она печатает для каждой точки среднее, разброс и длину принятия.

обычная генерация: 46.08 ± 0.21 мс/токен (разброс 0.4%)

А. глубина при фиксированном total_token = 64, top_k = 10

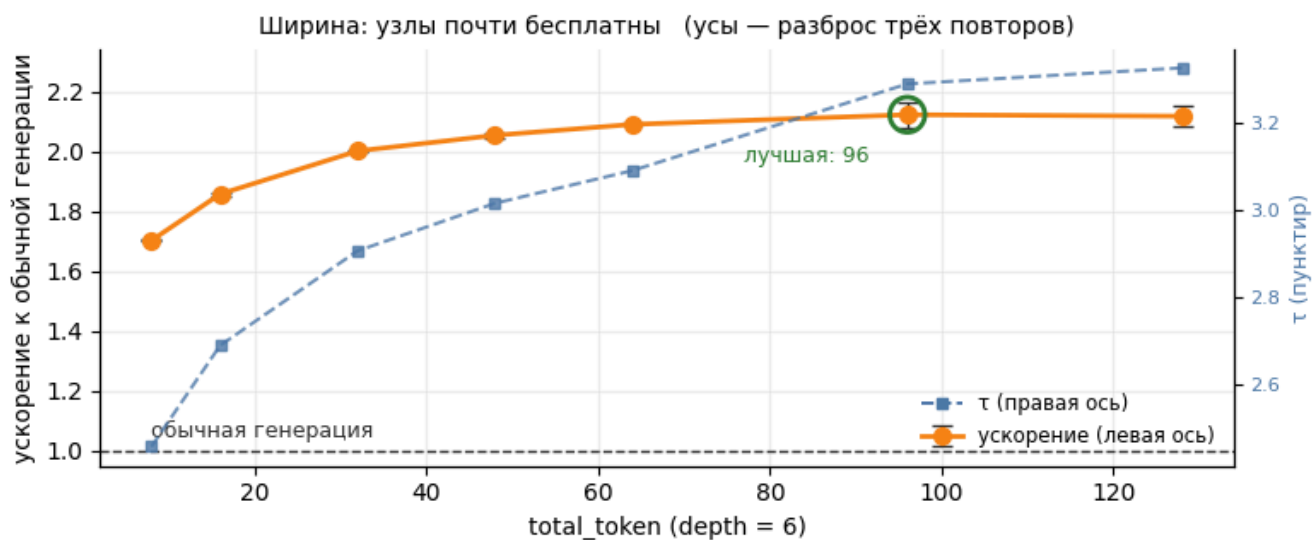
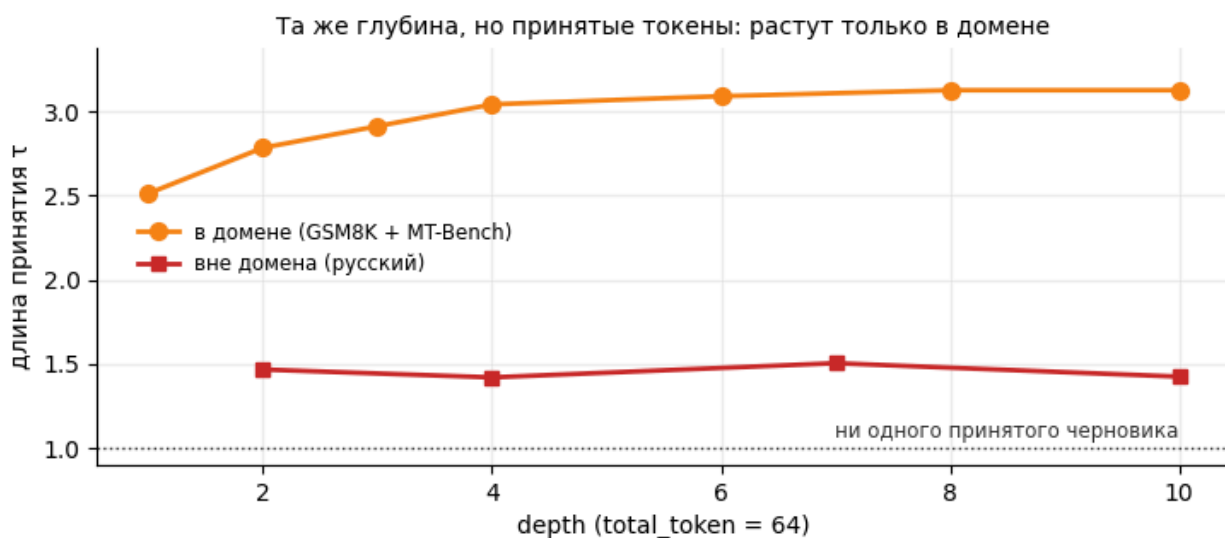
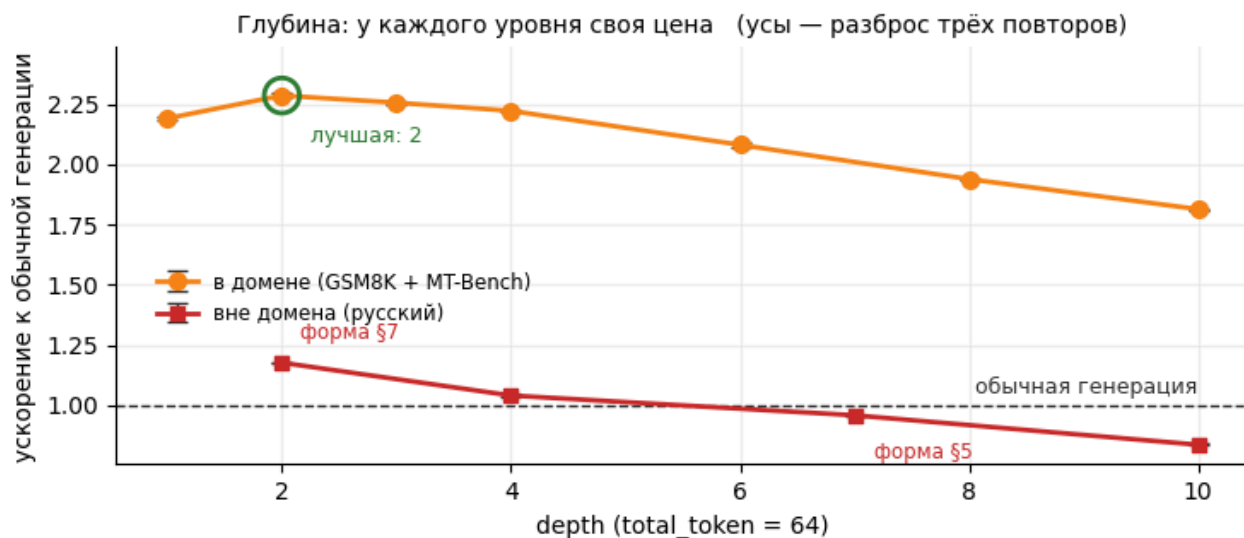
depth= 1:	21.04 ± 0.07 мс/ток		τ 2.51		ускорение 2.19x
depth= 2:	20.16 ± 0.09 мс/ток		τ 2.78		ускорение 2.29x
depth= 3:	20.43 ± 0.06 мс/ток		τ 2.91		ускорение 2.26x
depth= 4:	20.74 ± 0.06 мс/ток		τ 3.04		ускорение 2.22x
depth= 6:	22.15 ± 0.09 мс/ток		τ 3.09		ускорение 2.08x
depth= 8:	23.77 ± 0.04 мс/ток		τ 3.12		ускорение 1.94x
depth=10:	25.41 ± 0.05 мс/ток		τ 3.12		ускорение 1.81x

С. глубина на русскоязычном наборе (обычная генерация 46.20 мс/ток)

depth= 2:	39.25 ± 0.03 мс/ток		τ 1.47		ускорение 1.18x
depth= 4:	44.38 ± 0.26 мс/ток		τ 1.42		ускорение 1.04x
depth= 7:	48.18 ± 0.05 мс/ток		τ 1.50		ускорение 0.96x
depth=10:	55.23 ± 0.24 мс/ток		τ 1.42		ускорение 0.84x

В. размер дерева при фиксированной depth = 6, top_k = 10

total_token= 8:	27.06 ± 0.05 мс/ток		τ 2.46		ускорение 1.70x
total_token= 16:	24.80 ± 0.09 мс/ток		τ 2.69		ускорение 1.86x
total_token= 32:	23.01 ± 0.06 мс/ток		τ 2.91		ускорение 2.00x
total_token= 48:	22.42 ± 0.08 мс/ток		τ 3.01		ускорение 2.05x
total_token= 64:	22.03 ± 0.03 мс/ток		τ 3.09		ускорение 2.09x
total_token= 96:	21.70 ± 0.45 мс/ток		τ 3.29		ускорение 2.12x
total_token=128:	21.75 ± 0.33 мс/ток		τ 3.32		ускорение 2.12x



лучшая глубина (при `total_token = 64`): 2
лучший размер (при `depth = 6`): 96
τ с ростом дерева: 2.46 → 3.32, с ростом глубины: 2.51 → 3.12

совмещённая точка 96/2/10: 20.76 ± 0.36 мс/ток | τ 2.81 | ускорение 2.22x
она не лучше лучших точек из отдельных срезов — параметры взаимодействуют, берём лучшую проверенную конфигурацию

Вывод. Два среза ведут себя по-разному, и так, как предсказывает арифметика первого раздела. У глубины есть отчётливый оптимум, за которым она только вредит: длина принятия выходит на плато, а время растёт с каждым новым запуском черновой модели. Ширина почти бесплатна, пока проход плоский, поэтому её кривая поднимается и упирается в потолок, а не заваливается.

Красная линия на верхней панели — та же глубина, но на чужом домене, и она объясняет замедление из §5. Средняя панель показывает, почему: длина принятия там стоит на месте, около полутора на всех глубинах, тогда как в домене она растёт с каждым уровнем. За пределами своего домена черновик угадывает одинаково редко — что при двух уровнях, что при десяти. Значит, глубина покупает там одну только цену, и на глубоком дереве цена перевешивает: верхняя кривая уходит под единицу. То есть §5 измеряет на стартовой, глубокой форме и получает замедление, а на выбранной здесь мелкой те же запросы дают небольшой выигрыш. Домен решает, сколько метод может выиграть; глубина — сколько он может проиграть, вплоть до смены знака.

Мелкие глубины разделяет пара процентов — примерно столько же составляет дрейф между сессиями: от прогона к прогону победитель ходит между двойкой и тройкой, тогда как форма обеих кривых не меняется. Читать этот срез стоит по форме кривой, а не по тому, какая точка на ней вышла лучшей: замер показывает, во сколько обходится глубокий дефолт: около 10%. Какая из мелких глубин лучше, он не решает.

В репозитории есть режим, который подбирает размер дерева сам. Сравниваю его выбор с тем, что показали замеры выше.

авторский автоподбор выбрал бы `total_token = 60`
срез по ширине даёт пик около 96, но снят он при `depth = 6` — а на глубине, которую выбирает этот раздел (2), ширина насыщается заметно раньше.

Важнее другое: эвристика крутит единственную ручку, которая здесь ничего не решает. Она ищет `total_token` в диапазоне 40..60 и не трогает глубину, а платит на этой карте именно глубина — каждый уровень добавляет последовательный запуск черновика.

Дальше работаем с деревом `{'total_token': 64, 'depth': 2, 'top_k': 10}`

Вывод. Авторская эвристика автоподбора крутит единственную ручку, которая на этой карте ничего не решает: она перебирает размер дерева в диапазоне 40–60 узлов и не трогает глубину, а платит здесь именно глубина. Собственный оптимум среза по ширине лежит выше её потолка, но снят он при `depth = 6` и

перехода на мелкое дерево не переживает. Эвристику настраивали на других моделях и другом железе, и это тот случай, когда чужую настройку надо перемерить у себя, а не наследовать.

8. Куда уходит время цикла

У цикла четыре фазы: черновики (последовательные проходы головы), проверка (один проход целевой модели по всему дереву), выбор пути и всё остальное — копирование кэша, построение следующего дерева, служебная логика на стороне процессора.

Остаток получается вычитанием, но одним вычитанием я не ограничиваюсь: обёртки с синхронизацией сами стоят времени, поэтому рядом привожу время того же прогона без инструментации. Разница между двумя прогонами — цена самого измерения, и её видно.

Оборачиваю три фазы цикла таймерами и прогоняю дважды: с инструментацией и без неё.

цикл (с инструментацией): 56.2 мс, приносит $\tau = 2.52$

цикл (без инструментации): 55.8 мс ← цена самих замеров 0.3 мс

черновики 9.0 мс (16.0%)

проверка 45.4 мс (80.9%)

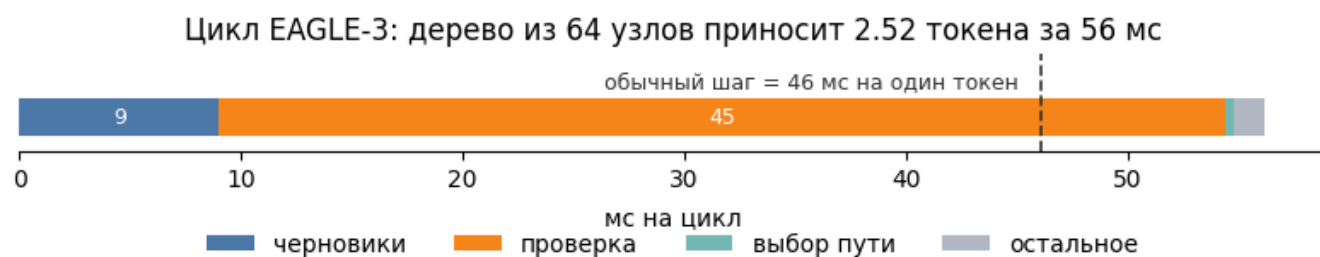
выбор пути 0.4 мс (0.7%)

остальное 1.4 мс (2.4%)

обычный шаг: 46.1 мс на 1 токен

проверка дерева из 64 узлов: 45.4 мс = 0.99 обычного шага

а проход целевой модели из §1 на 64 токенах: 47.3 мс — сходится



Вывод. Проверка занимает около 80% цикла и стоит практически столько же, сколько обычный шаг ради одного токена. Только приносит она не один токен, а два с половиной — дерево здесь уже в той форме, которую выбрал §7. Черновая модель забирает около 15%, выбор пути и копирование кэша вместе — пара процентов. Та же цена, больше результата.

9. Температура: где проверяется главная гарантия метода

Вторая половина обещания — то, что метод ничего не портит.

При жадном декодировании «неухудшение качества» — почти тавтология: принимается только тот черновик, который совпал с `argmax` целевой модели. Содержательная гарантия формулируется для сэмплирования: спекулятивное декодирование обязано выдавать в точности то же распределение, что и обычное сэмплирование из целевой модели.

Здесь нужна точность, потому что реализация в репозитории отличается от того, что описано в классических статьях. У [Leviathan](#) и [Chen](#) черновик принимается с вероятностью $\min(1, \frac{p}{q})$, где p — вероятность по целевой модели, q — по черновой, а при отказе сэмплируют из нормализованной разности. В коде EAGLE (`evaluate_posterior`, ветка с `logits_processor`) стоит `qx = 1.0`: кандидат принимается с вероятностью $p(x)$, при отказе $p(x)$ обнуляется, распределение перенормируется, и пробуются следующий уникальный кандидат из дерева.

Эта схема тоже несмещённая, и убедиться в этом можно в одну строку. Вероятность того, что кандидат a будет отвергнут, а на его месте окажется токен b , равна

$$(1 - p(a)) \cdot \frac{p(b)}{1 - p(a)} = p(b)$$

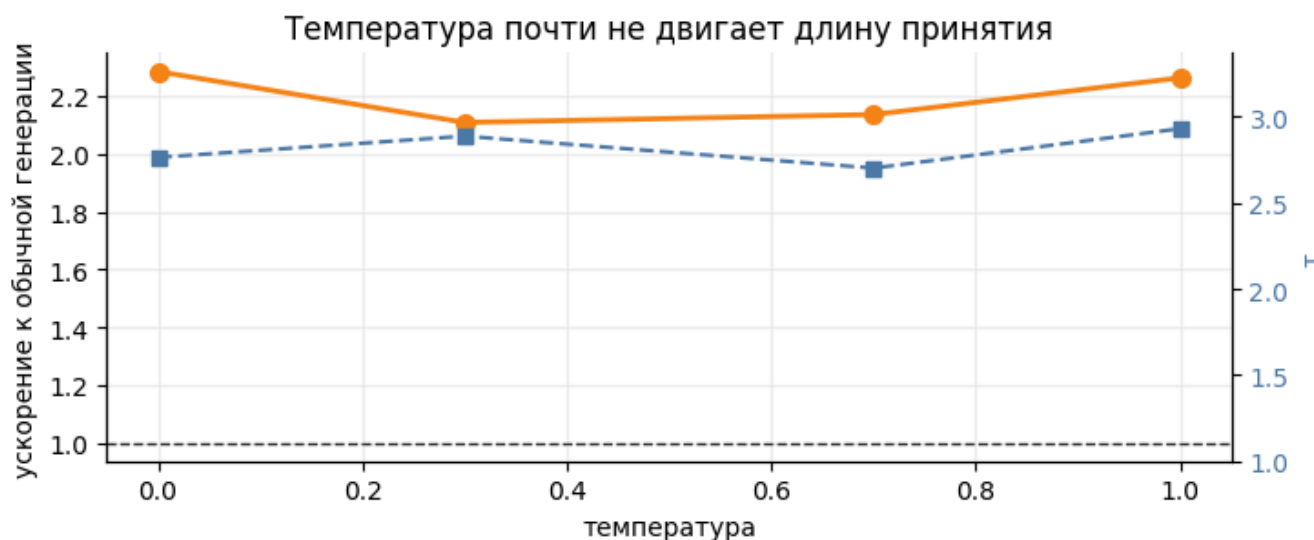
Первый множитель — вероятность отказать кандидату, второй — вероятность выбрать b из перенормированного распределения; множитель отказа сокращается с нормировкой.

Индукцией по кандидатам получается то же самое для любого их числа, так что итоговое распределение совпадает с p при любой стратегии предложения: черновая модель влияет на скорость, но не на то, что получится.

Всё это доказано для точной арифметики. В fp16 логиты по дереву и логиты по одному токenu считаются разными последовательностями операций, и совпадать бит-в-бит они не обязаны. Поэтому измеряю обе вещи сразу: совпадают ли распределения и насколько расходятся логиты.

Прогоняю одни и те же запросы при четырёх значениях температуры и смотрю, как меняются длина принятия и скорость.

T=0.0:	τ 2.76	20.17 мс/ток	ускорение 2.28x
T=0.3:	τ 2.89	21.85 мс/ток	ускорение 2.11x
T=0.7:	τ 2.70	21.57 мс/ток	ускорение 2.14x
T=1.0:	τ 2.93	20.36 мс/ток	ускорение 2.26x



Вывод. Ожидание было такое: чем выше температура, тем реже черновик угадывает и тем меньше выигрыш. У авторов оно подтверждается — для LLaMA-3.1-8B они приводят 4.44x при $T = 0$ против 3.45x при $T = 1$, то есть падение примерно на 20%. Мой замер такого падения не показывает: длина принятия и ускорение гуляют в пределах нескольких процентов без внятного тренда. Считать это опровержением я не стал бы — здесь два запроса без повторов, целевая модель почти впятеро меньше и генерация короче, так что чувствительности может не хватать. Что можно утверждать: обвала при полном сэмплировании не происходит, метод остаётся примерно вдвое быстрее обычной генерации.

Собираю по 500 независимых продолжений каждым способом. Диапазоны случайных зёрен у них разные: выборки должны быть независимыми, иначе сравнение окажется парным и покажет сходство там, где его нет.

собрано за 493 с: EAGLE 500, обычная 500, контрольная 500

Статистику здесь легко испортить.

Сравнивать эмпирические распределения по расстоянию полной вариации (TV) недостаточно: у двух конечных выборок из одного и того же распределения TV всё равно больше нуля, и тем больше, чем шире носитель. Значит, нужна нулевая полоса — сколько TV набегает просто от того, что выборки конечны.

Бутстрэп из наблюдаённой выборки для этого не годится: пересэмплирование из пула даёт больше совпадений, чем два настоящих независимых замера, полоса выходит слишком узкой, и тест объявляет расхождением любую разницу. Правильный инструмент —

перестановочный тест: если распределения совпадают, метки выборок обменимы, поэтому перемешивание меток между объединёнными выборками даёт точную нулевую статистику без предположений о носителе.

И чтобы не судить об исправности теста по тем же данным, которые он проверяет, рядом идёт контроль: две независимые выборки обычной генерации. Разницы на них тест найти не должен.

КОНТРОЛЬ – обычная против обычной (тест обязан НЕ найти разницы):

совместное распределение 5 токенов: TV=0.1440, порог=0.1940, $p=0.865$ тест исправен

ТЕСТ – EAGLE против обычной:

совместное распределение 5 токенов: TV=0.2060, порог=0.2060, $p=0.053$

различных последовательностей: обычная 65, EAGLE 65

ВЕРДИКТ: распределения неразличимы

по отдельным позициям (позиция 0 берётся из логитов целевой модели напрямую, позиции 1 и дальше уже проходят через спекулятивный приём и отклонение):

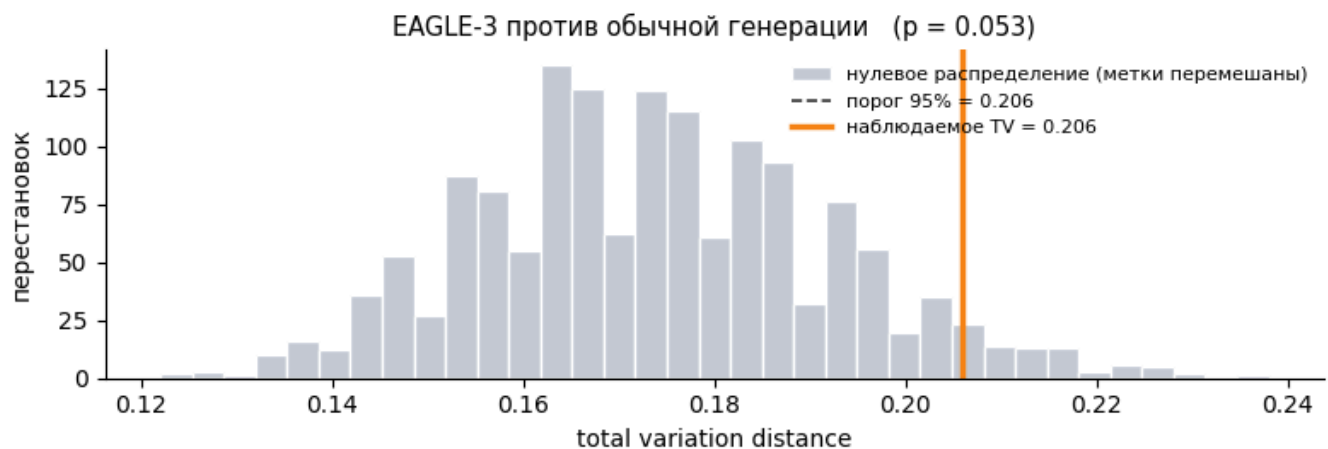
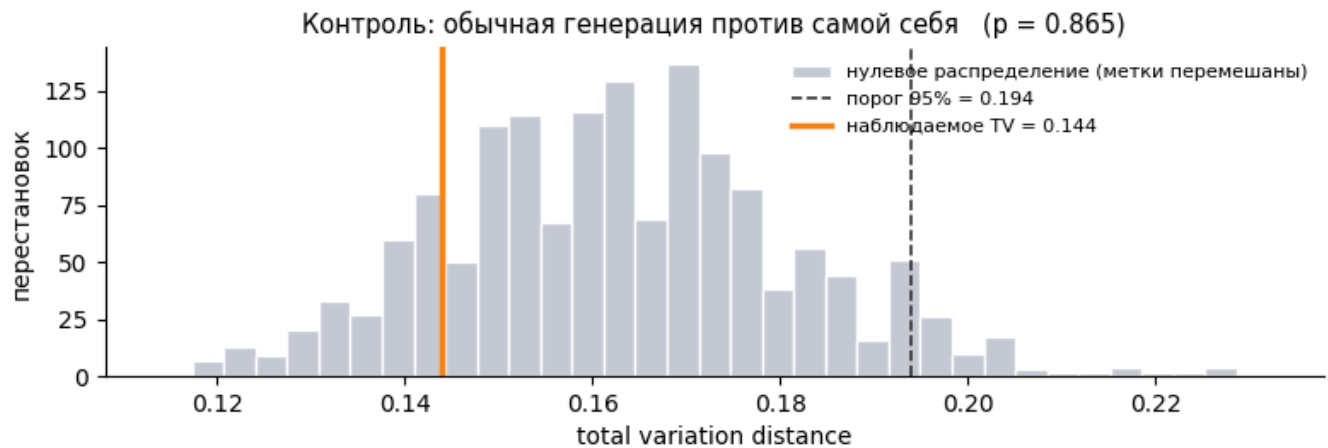
поз 0: EAGLE $p=0.033$!! | контроль $p=0.827$ | различных токенов 10

поз 1: EAGLE $p=0.528$ ok | контроль $p=0.410$ | различных токенов 3

поз 2: EAGLE $p=0.003$!! | контроль $p=0.192$ | различных токенов 12

поз 3: EAGLE $p=0.498$ ok | контроль $p=0.482$ | различных токенов 11

поз 4: EAGLE $p=0.118$ ok | контроль $p=0.507$ | различных токенов 16



частоты первого токена (обычная | контрольная | EAGLE):

'book':	174		174		160
'apple':	137		141		180
'dog':	72		83		65
'cat':	68		60		51
'car':	35		27		29
'bird':	6		4		6
'tree':	3		6		4
'door':	3		2		1

Вывод. Сверху контроль: две независимые выборки обычной генерации, и тест не находит между ними разницы — значит, он не склонен видеть расхождения там, где их нет. Снизу сравнение с EAGLE, и наблюдаемое расстояние укладывается в то же нулевое распределение. Укладывается впритык: p выходит к самой границе, запаса этот тест не оставляет. Поэтому вопрос закрывает не он, а прямое сравнение распределений ниже.

Откуда берётся остаточная разница

Совместный тест разницы не находит, но по двум позициям из пяти p оказывается низким. Прежде чем считать, что метод всё-таки сдвигает распределение, посмотрю на позицию 0: первый новый токен в обоих случаях берётся напрямую из логитов целевой модели, спекулятивный приём в нём вообще не участвует. Если разница есть уже там, значит дело в арифметике, а правило приёма ни при чём.

Здесь можно обойтись без теста: я перехватываю тензор вероятностей, из которого в каждом случае берётся первый токен, и сравниваю два распределения напрямую.

Ячейка ниже ставит хук на выбор токена и печатает обе таблицы вероятностей рядом.

размер словаря: 151936

TV между распределениями, из которых берётся первый токен: 0.00000

максимальная разница вероятности одного токена: 0.00000

бит-в-бит одинаковы: True

токен	обычная	EAGLE	разница
'apple'	0.33350	0.33350	+0.00000
'book'	0.31812	0.31812	+0.00000
'dog'	0.15259	0.15259	+0.00000
'cat'	0.11169	0.11169	+0.00000
'car'	0.05792	0.05792	+0.00000
'tree'	0.01072	0.01072	+0.00000
'bird'	0.00714	0.00714	+0.00000
'door'	0.00302	0.00302	+0.00000

Распределения совпадают бит-в-бит — остаточная разница в тесте выше объясняется конечностью выборки и множественными сравнениями.

10. Совпадение с обычной генерацией при $T = 0$

В жадном режиме требование строже: совпадать должно не распределение, а сама последовательность. EAGLE обязан выдать ту же цепочку токенов, что и обычный цикл. Сравниваю выдачи токен за токеном. Если расхождения найдутся, измеряю их величину и смотрю, где именно они возникают.

Гипотеза для проверки простая. Обычный шаг обрабатывает один токен, спекулятивный — дерево из десятков узлов; это разные формы тензоров, разные CUDA-ядра и разный порядок суммирования в fp16. Если в какой-то позиции два лучших кандидата почти равны по логиту, порядок суммирования решает исход. Значит, в точках расхождения разрыв между top-1 и top-2 должен быть аномально мал по сравнению с обычными позициями. Это проверяемое предсказание, и его легко опровергнуть: если разрывы в точках расхождения обычные, дело не в численной погрешности, а в реализации.

Сравниваю выдачу двух способов токен за токеном на десяти запросах.

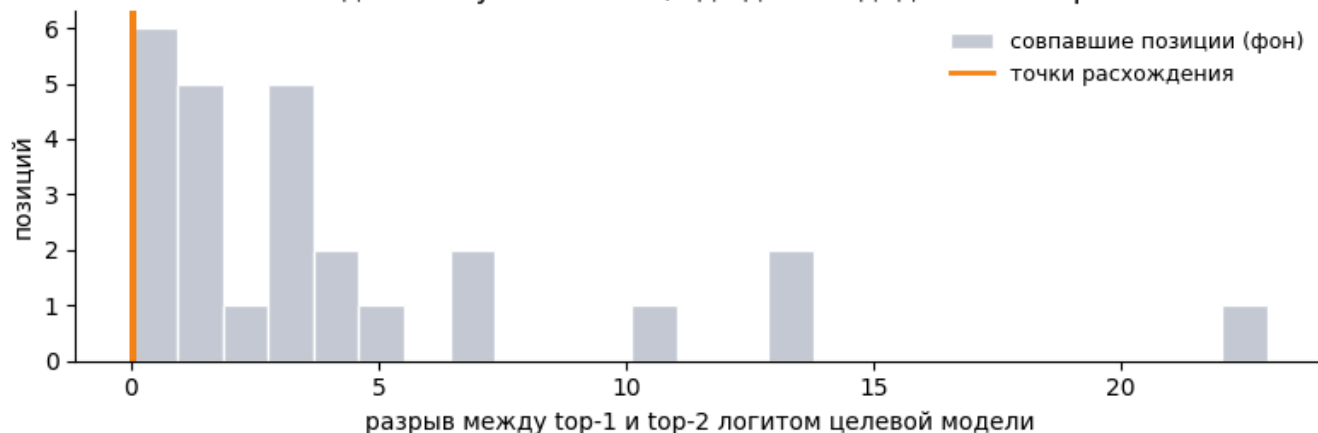
проверено 10 запросов, 2219 токенов
полное совпадение: 6 из 10 запросов
расхождений: 4 = 1.80 на 1000 токенов

Теперь проверяю саму гипотезу: беру позиции, где выдачи разошлись, и смотрю на разрыв между двумя лучшими кандидатами целевой модели. Для сравнения беру тот же разрыв на позициях, где всё совпало.

запрос: MT-Bench — 'Compose an engaging travel blog post about a recent trip to '
совпало 178 токенов, дальше разошлись
обычная: ' left me in tears of appreciation.\n\nI'
EAGLE: ' truly brought me into the heart of Hawaiian'

разрыв top1-top2 в точке расхождения: 0.0000
медиана разрыва на совпавших позициях: 2.8359
перцентиль точки расхождения в фоне: 0%

Расхождения случаются там, где два кандидата почти равны



Вывод: разрывы в точках расхождения лежат у самого нуля, тогда как обычная позиция уверенно отделена. Это ровно та картина, которую предсказывает гипотеза о порядке суммирования в fp16, а не ошибка в правиле приёма: гарантия `losslessness` формулируется для точной арифметики.

Вывод. Гипотеза была фальсифицируемой: если расхождения вызваны численной погрешностью, они обязаны приходиться на позиции, где два кандидата почти неразличимы. Так и вышло — точки расхождения прижаты к нулю, тогда как обычная позиция отделена уверенным зазором. Это следствие порядка суммирования в fp16, а не изъян правила приёма.

11. Тот же метод, цель вдвое крупнее

Формула из §1 подсказывает предсказание, которое ноутбук пока не проверял. Рассуждают обычно так: черновая голова маленькая и стоит примерно одинаково, а шаг целевой модели растёт вместе с её размером — значит, на большей цели тот же метод должен окупаться лучше. Рассуждение проверяемое, и его лучше проверить, чем повторять. Qwen3-4B — самая крупная пара, которая влезает в бесплатную T4 в fp16: 8B весит 16.4 ГБ, больше, чем есть у карты. В другую сторону сравнивать не с чем: головы под Qwen3-0.6B AngelSlim не публиковала, так что 1.7B здесь — нижняя граница измеримого.

Порядок такой. Сначала перемеряю пару 1.7B на небольшом подмножестве при той форме дерева, которую выбрал §7, и полностью выгружаю её с карты. Потом грузю пару 4B из прикреплённых моделей и прогоняю то же подмножество, то же дерево, те же правила равной работы.

Прежде чем смотреть числа — про сам запуск. Официальный репозиторий эту голову в опубликованном виде не загружает: внимание Qwen3-4B шире её скрытого размера (32 головы внимания $\times 128 = 4096$ против 2560), а официальный код черновика выводит ширину черновой головы из скрытого размера. Ячейка окружения правит три строки `cnets.py`, чтобы эта ширина бралась из `head_dim` в конфиге. Для пары 1.7B это ничего не меняет, и это единственная правка официального кода во всём ноутбуке.

пара 1.7B при дереве {'total_token': 64, 'depth': 2, 'top_k': 10}:

в домене : обычная 45.9 мс/ток | EAGLE 18.9 | τ 2.96 ± 0.00 | ускорение 2.42x ± 0.0
1 по 3 повторам

вне домена: обычная 46.0 мс/ток | EAGLE 41.5 | τ 1.36 ± 0.00 | ускорение 1.11x ± 0.0
01 по 3 повторам

после выгрузки занято на GPU: 0.01 ГБ

/kaggle/input/models/qwen-lm/qwen-3/transformers/4b/1

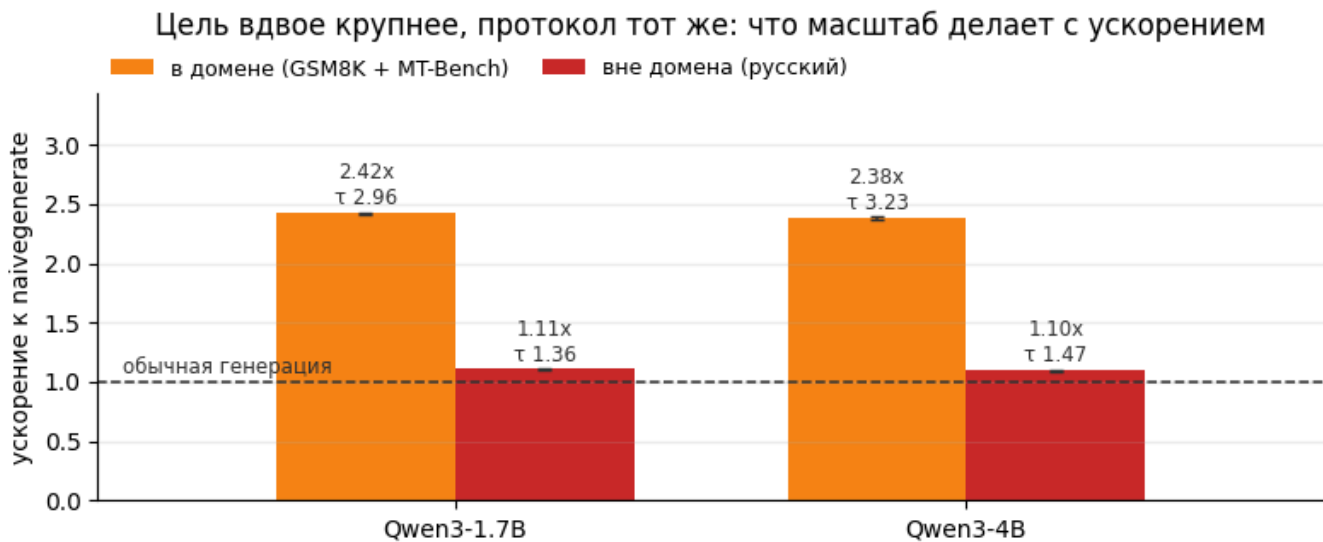
/kaggle/input/models/georgymamarin/qwen3-4b-eagle3-draft-head/pytorch/bf16/1

пара 4B загружена за 71 с | цель 4.02 млрд параметров | GPU 8.65 ГБ

пара 4B при дереве {'total_token': 64, 'depth': 2, 'top_k': 10}:

в домене : обычная 59.7 мс/ток | EAGLE 25.1 | τ 3.23 ± 0.00 | ускорение 2.38x ± 0.0
1 по 3 повторам

вне домена: обычная 59.7 мс/ток | EAGLE 54.3 | τ 1.47 ± 0.00 | ускорение 1.10x ± 0.0
01 по 3 повторам



Вывод. Две цели, один протокол — и предсказание сбывается ровно наполовину.

Длина принятия растёт вместе с размером цели, и растёт заметно: около 9% в пользу пары 4B, и в домене, и вне его. Жадное декодирование фиксирует последовательность токенов, поэтому внутри прогона все повторы возвращают одну и ту же длину принятия до третьего знака; усы на графике — чистый разброс времени. А ускорение за ней не идёт: разница между парами не выходит за 2–3%, и её знак меняется от прогона к прогону. Величина без шума говорит, что голова стала угадывать лучше; величина с шумом не может сказать, изменилось ли вообще что-нибудь. Удвоение цели купило длину принятия и, насколько эта карта способна различить, не купило скорости.

Механизм читается по сырым замерам времени; ускорения его прячут. И здесь важно не путать цикл с токеном. Обычный шаг на большей цели дорожает. Полный спекулятивный цикл — один заход черновика плюс одна проверка дерева — дорожает быстрее: проверка тащит те же расширившиеся веса, что и обычный шаг, а черновик перестал быть бесплатным. Те 9%

лишних токенов, которые приносит цикл, эту разницу почти целиком и съедают. Остаётся пара процентов — того же размера, что и дрейф между прогонами, отчего её знак и гуляет.

Ломается, стало быть, предпосылка: цена черновой головы не фиксирована. Эта голова выросла вместе с целью, со 137 до 218 млн параметров, потому что её скрытый размер и внимание расширяются следом за моделью. Как доля цели она действительно ужалась, с 8.0% до 5.4%, но знаменатель формулы платит не за долю, а за абсолютное время черновика.

Столбики вне домена указывают ещё на один открытый вопрос, но не закрывают его. Ни одна пара здесь не проваливается ниже 1.0 — при том что в §5 на тех же русскоязычных запросах метод проигрывал обычной генерации. Сказать этот замер может одно: длина принятия против §5 почти не сдвинулась, то есть более короткое подмножество не стало легче, и поменялась цена цикла; частота, с которой черновик угадывает, осталась прежней. Указывает это на глубину дерева, и §7 догадку проверяет: там тот же срез по глубине снят на русскоязычном наборе, и кривая пересекает единицу, пока длина принятия держится около полутора на всех глубинах.

key_numbers.json записан (15 групп, scaling: есть)

12. Что не сработало

Тупики и грабли, на которые ушло время. Перечисляю, чтобы их не повторяли.

P100 вместо T4. Kaggle по умолчанию выдаёт под `enable_gpu` карту P100, а предустановленный там `torch 2.10+cu128` не поддерживает `sm_60`: падает с `CUDA error: no kernel image is available`. Лечится выбором T4 (в API — поле `machine_shape: NvidiaTeslaT4`).

Установка репозитория через pip. `setup.py` в EAGLE тянет `torch==2.0.1` и `transformers==4.46.2` — на свежем окружении это ломает всё. Правильный путь: клонировать и добавить в `sys.path`, а версию `transformers` фиксировать отдельно (`4.53.1`; предустановленная на Kaggle 5.x несовместима с модифицированными modeling-файлами репозитория).

Глубокое дерево по умолчанию. Стартовая форма `60/7/10` и авторские `32/8/4` роднит одно: глубина в семь-восемь уровней. Абляция §7 показывает, что на этой карте она не окупается — длина принятия выходит на плато уже к четвёртому уровню, а платить приходится за каждый. Сам я `32/8/4` не измерял: срез по глубине в §7 измеряет глубину при фиксированных `total_token` и `top_k`, то есть трогает только одну из трёх ручек. Поэтому чисел именно для этой формы я не привожу — вывод здесь про глубину, и конкретная конфигурация его не меняет.

Погоня за разницей в проценты. Первая версия этого ноутбука выбирала конфигурацию по разнице 2.42× против 2.43× при одном прогоне на точку. Повторные замеры показали

разброс порядка процента: разница была шумом. Отсюда повторы (по три на точку) и усы на всех графиках.

Наращивание дерева до бесконечности. t продолжает расти и после 96 узлов, а скорость — нет: выигрыш съедается стоимостью построения самого дерева. Узкое место переезжает с проверки на черновую модель.

Нулевая полоса из бутстрэпа. Первая версия теста при $T = 1$ строила нулевую полосу пересэмплированием из наблюдаённой выборки. При разреженном носителе это систематически занижает полосу: бутстрэп берёт из пула в несколько сотен значений и даёт больше совпадений, чем два настоящих независимых замера. Тест уверенно объявил расхождение там, где его нет. Перестановочный тест этим не страдает, а контроль на двух выборках обычной генерации ловит такую ошибку сразу: на сломанном тесте он тоже показывал расхождение.

Погоня за низкими p -value по отдельным позициям. Даже с корректным тестом две позиции из пяти дали $p < 0.05$. Объявить это сдвигом распределения заманчиво, но при десяти сравнениях (пять позиций на два теста) и $\alpha = 0.05$ в среднем ожидается одно ложное срабатывание на два таких прогона, так что одни p -value вопрос не закрывают. Для позиции 0 его закрыл прямой замер: распределения совпадают бит-в-бит, и расхождению там взяться неоткуда. Для остальных позиций прямое сравнение невозможно в принципе — там распределение зависит от уже выпавшего префикса, а он у двух прогонов свой. Остаётся списать эти p -value на множественные сравнения: доказать сверх этого нечем.

Тест распределения на «удобном» запросе. Первая попытка проверить неизменность распределения при $T = 1$ использовала запрос, у которого продолжение почти детерминировано: TV вышло ровно 0 при нулевой полосе $[0, 0]$. Формально совпало, содержательно — тест ничего не проверял. Нужен запрос с реальной энтропией, иначе проверяется пустота.

13. Выводы

1. Спекулятивное декодирование работает потому, что декодирование **memory-bound**.

На T4 проход целевой модели не дорожает до 192 токенов, поэтому проверка целого дерева стоит примерно как генерация одного токена. Всё остальное в методе — попытки набрать побольше принятых токенов за этот один проход.

2. Ускорение определяется длиной принятия, а её задают домен и форма дерева

вместе. Одна и та же черновая модель даёт разброс от двух с половиной раз на математике и коде до замедления на русскоязычных запросах — но замедление снято на глубоком стартовом дереве. На форме, которую выбирает §7, те же запросы дают небольшой выигрыш при почти неизменной длине принятия, и §11 подтверждает это на второй паре моделей. Домен задаёт, сколько метод может выиграть; глубина дерева —

сколько он может проиграть. Длину принятия надо измерять на своём распределении запросов; чужие цифры не переносятся.

3. **Ширина дерева дешевле глубины.** Ширина добавляет узлы в тот же проход проверки, глубина — новые последовательные запуски черновой модели. Авторская эвристика автоподбора при этом крутит только число узлов и вовсе не трогает глубину.
4. **Метод не портит качество, и это проверяемо тремя способами.** При $T = 0$ выдача совпадает потоленно, а редкие расхождения приходятся строго на позиции, где два кандидата почти неразличимы по логиту: разрыв там неотличим от нуля против типичных 2.8 логита. При $T = 1$ перестановочный тест не отвергает равенство распределений; контроль на двух выборках обычной генерации показывает, что тест не видит расхождения там, где его заведомо нет — на предыдущей, сломанной версии теста этот же контроль расхождение показывал. Третий способ — прямое сравнение: распределение, из которого в обоих случаях берётся первый токен, совпадает бит-в-бит. На последующих позициях так сравнить нельзя: там распределение зависит от уже выпавшего префикса.
5. **Масштаб покупает длину принятия, а не скорость — по крайней мере на этом удвоении.** Интуицию «на большей цели черновая голова окупается лучше» §11 доводит до замера на Qwen3-4B: τ растёт примерно на 9%, а ускорение не выходит за пределы шума, потому что цена этой головы не фиксирована: она растёт вместе со своей целью. Возобновится ли рост на 8B и дальше, этот замер не скажет: такая пара в бесплатную T4 не помещается.
6. **Заявленные ускорения всегда надо читать вместе с базовым вариантом.** Обычный шаг здесь работает на четверти пропускной способности памяти; часть измеренного выигрыша — амортизация этих накладных расходов, а не победа над памятью. На оптимизированном стеке тот же метод даёт меньше — поэтому независимые замеры на vLLM и скромнее авторских.

Чего здесь нет. Батч больше единицы (реализация в репозитории работает только с `batch = 1`, а именно батчинг сильнее всего съедает выигрыш спекуляции), длинные контексты и обучение своей головы под домен. Последнее — самое интересное продолжение, учитывая пункт 2.

14. Прогнать у себя, что читать дальше, лицензии

Форкните ноутбук и поменяйте две строки из четырёх, описанных в §0:

```
BASE_MODEL = "Qwen/Qwen3-4B"           # любая целевая модель, под
которую есть голова
EA_MODEL    = "AngelSlim/Qwen3-4B_eagle3" # список голов — в README
репозитория EAGLE
```

Про размер — как раз к примеру выше. §5 держит на карте вторую, независимую копию цели: без неё нечем замерить штатный `generate`. Рядом с парой 1.7B она помещается свободно, рядом с парой уровня 4B — уже нет: §11 потому и выгружает первую пару перед загрузкой

второй. Так что двухстрочный форк проверен примерно до 2B; для цели покрупнее либо уберите базовую линию штатного `generate` в §5, либо выгружайте предыдущую пару перед загрузкой следующей, как это делает §11.

Чтобы измерить длину принятия на своих запросах (пункт 2 выводов), добавьте свой набор в словарь `BENCH` в §5 — дальше все таблицы и графики построятся по нему. Где именно проходит точка окупаемости, зависит от дерева, поэтому переносить надо не число, а связку. Одна и та же длина принятия — около полутора токенов за цикл — на глубоком стартовом дереве стоит ровно на окупаемости (§5), а на мелком, которое выбирает §7, уходит в небольшой плюс (§11). Низкая длина принятия и глубокое дерево вместе превращают спекуляцию в проигрыш, и знать это надо до внедрения, а не после.

Ячейка ниже печатает текущую конфигурацию и то, что в ней можно менять.

```
# Текущая конфигурация разбора. Меняется в первой ячейке ноутбука.
print(f'BASE_MODEL = "{BASE_MODEL}"' + (' if BASE_PATH == BASE_MODEL else f'    <- вз'
print(f'EA_MODEL    = "{EA_MODEL}"' + (' if EA_PATH == EA_MODEL else f'    <- взято с ,
print(f'дерево      = {MAIN_TREE}')
print(f'наборы      = {list(BENCH)}')
print()
print("Чтобы прогнать на своей модели: заменить две первые строки и Run All.")
print("Чтобы прогнать на своих задачах: добавить свой список в BENCH (§5).")
```

```
BASE_MODEL = "Qwen/Qwen3-1.7B"    <- взято с диска: /kaggle/input/models/qwen-lm/qwen-
3/transformers/1.7b/1
EA_MODEL    = "AngelSlim/Qwen3-1.7B_eagle3"    <- взято с диска: /kaggle/input/models/g
eorgymamarin/qwen3-1-7b-eagle3-draft-head/pytorch/fp16/1
дерево      = {'total_token': 64, 'depth': 2, 'top_k': 10}
наборы      = ['MT-Bench', 'GSM8K', 'HumanEval', 'Alpaca', 'Русский']
```

Чтобы прогнать на своей модели: заменить две первые строки и Run All.

Чтобы прогнать на своих задачах: добавить свой список в BENCH (§5).

Что читать дальше

Как метод дошёл до нынешнего вида

- [Leviathan et al., 2211.17192](#) — исходная идея и доказательство несмещённости через rejection sampling. В приложении A.1 лежит доказательство, если хочется проверить самому.
- [Chen et al., 2302.01318](#) — независимая параллельная работа, полезна другим взглядом на то же доказательство.
- [EAGLE-1, 2401.15077](#) — почему предсказывать признак легче, чем токен.
- [EAGLE-2, 2406.16858](#) — динамическое дерево; отсюда и берутся `total_token`, `depth`, `top_k` из §7.
- [EAGLE-3, 2503.01840](#) — разбираемая статья: отказ от `l_fea`, слияние трёх уровней, training-time test.

- [HASS, 2408.15766](#) — ближайший предок идеи раскатывать черновую модель на собственных выходах при обучении.

На чём держится этот разбор

- [LLM Scaling Week](#) — лекция 1 «Deep Learning Arithmetic» (Михаил Хрущёв) даёт ту самую арифметику байтов и FLOP, на которой стоит §1, а лекция 5 «Inference Challenges» (Роман Горб) — картину узких мест инференса целиком.
- [Курс NLP Школы анализа данных](#) — week03 про KV-кэш (фундамент §2) и week05 про большие языковые модели.

Как это выглядит в промышленной эксплуатации

- «[Performance or Illusion?](#)», [2601.11580](#) — независимая репликация на vLLM. Главная мысль: ускорение падает с ростом батча, потому что батчинг сам по себе загружает вычислители, и на спекуляцию их уже не остаётся.

Код и веса

- [SafeAILab/EAGLE](#) — официальная реализация, коммит `cb7e0841`, лицензия Apache 2.0. Ноутбук использует её как есть, за единственным исключением: три строки `cnets.py`, которые правит ячейка окружения в §0 — без них §11 не загрузит голову Qwen3-4B. Смотреть надо `eagle/model/cnets.py` (построение дерева) и `eagle/model/utils.py` (проверка и работа с KV).
- [AngelSlim/Qwen3-1.7B_eagle3](#) — черновая голова, обученная командой [AngelSlim](#) (Tencent). Она не авторская: замеры самой команды дают длину принятия 1.8–3.5, что сходится с моей; ускорение 1.4–1.9× они приводят к своей базовой линии. Та же команда обучила и [AngelSlim/Qwen3-4B_eagle3](#) — голову, на которой считает §11.
- [Qwen/Qwen3-1.7B](#) — целевая модель, лицензия Apache 2.0; в §11 к ней добавляется [Qwen/Qwen3-4B](#) под той же лицензией.
- Наборы вопросов (MT-Bench, GSM8K, HumanEval, Alpaca) взяты из каталога `eagle/data` того же репозитория — это те же наборы, на которых считает статью.

English summary

EAGLE-3 speculative decoding, reproduced on Qwen3-1.7B with a free Kaggle T4. The draft head is `AngelSlim/Qwen3-1.7B_eagle3` (the authors ship no head for this target); the code is the official `SafeAILab/EAGLE` at commit `cb7e0841`.

Everything is measured, not quoted. A target forward pass stays flat from 1 to ~192 tokens, so verifying a whole draft tree costs about as much as generating one token — that is the entire mechanism. Measured against the repository's own vanilla loop: 2.3× on the English benchmarks the paper uses (MT-Bench, GSM8K, HumanEval, Alpaca) with acceptance length $\tau \approx 3.4$, and $\approx 0.95\times$ — an actual slowdown — on Russian prompts, where the community-trained head is out of its domain. That slowdown needs the deep starting tree, though: at the shallow shape §7 picks, the same

prompts come out slightly ahead. Branching adds 39% to τ over a plain chain of the same depth. Depth beyond 2–3 costs more than it returns: τ plateaus at 3.1 while time keeps growing. Output is unchanged: token-identical under greedy decoding except at positions where the top-2 logit gap is numerically zero, and a permutation test finds no distribution shift at $T = 1$.

Read this walkthrough in English: [English version](#) — the same measurements, written for a wider audience.

Авторство и благодарности

Автор разбора — [Georgy Mamarin](#). Постановка вопросов, замеры, код ноутбука и текст мои; метод, реализация, веса черновой головы и наборы вопросов принадлежат авторам, перечисленным выше.

Отдельная благодарность команде [SafeAILab](#) — за открытую реализацию, которую разбор запускает как есть, если не считать трёх строк, без которых §11 не загрузит голову Qwen3-4B; команде [AngelSlim](#) в Tencent — за публичные черновые головы под Qwen3-1.7B и Qwen3-4B; Kaggle — за бесплатную T4, на которой всё посчитано.

Код ноутбука — Apache 2.0, модели и веса остаются под своими лицензиями.

Если воспроизведёте на другой модели или найдёте ошибку в замере, напишите в комментариях: обе новости одинаково полезны.